BÀI TẬP THỰC HÀNH SỐ 1

Objective

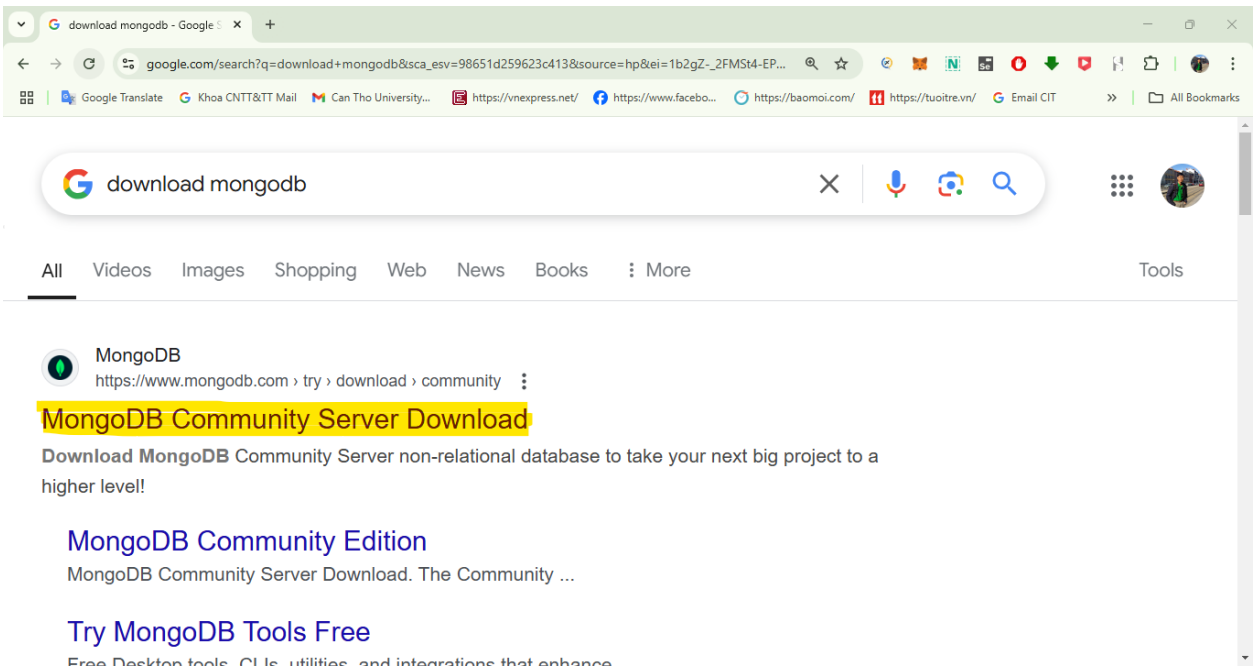
- Installation
- Query

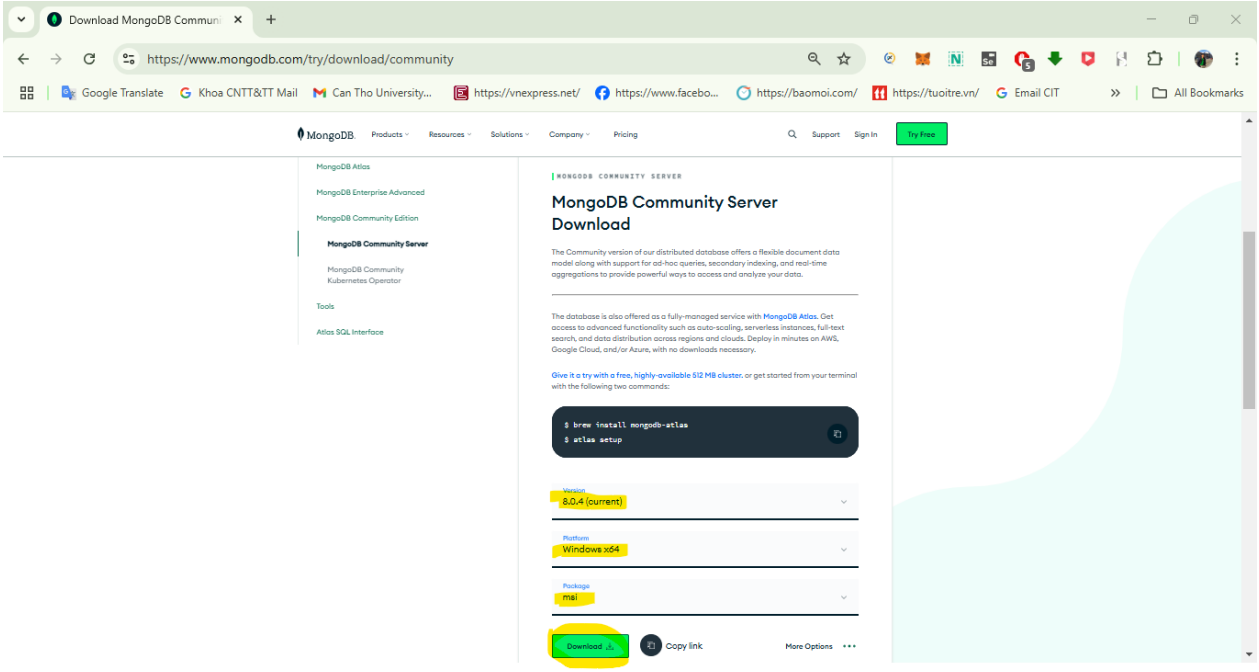
INSTALL AND CONFIGURE MONGODB AND MONGODB COMPASS

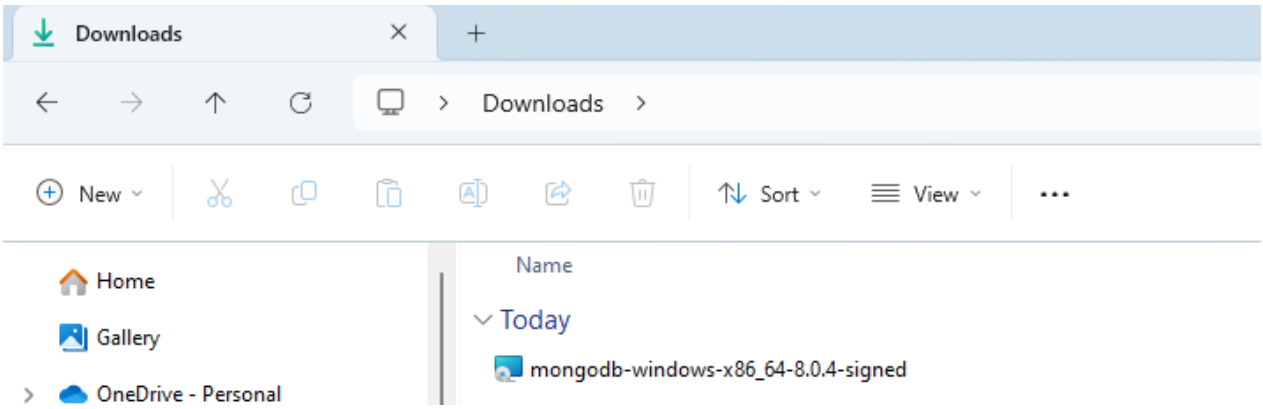
I.1. Install and Configure MongoDB

Download MongoDB tại: https://www.mongodb.com/try/download/community.

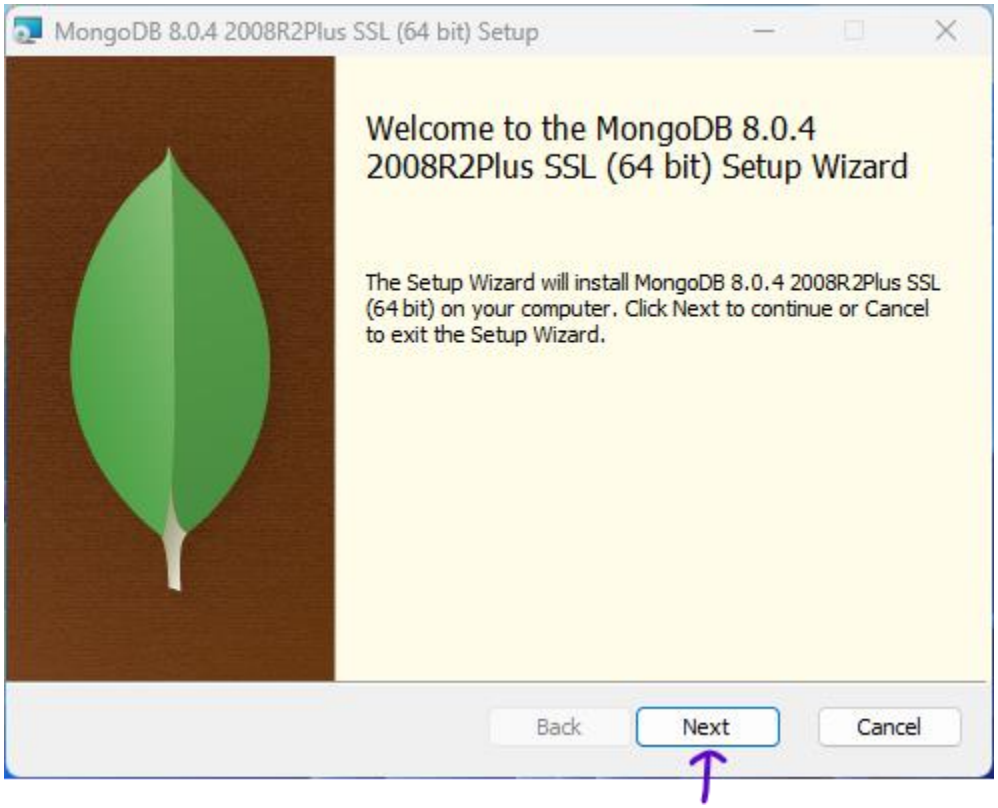
Proceed with normal installation (note: SHOULD choose the installation folder as C:\mongodb instead of going to Program Files)



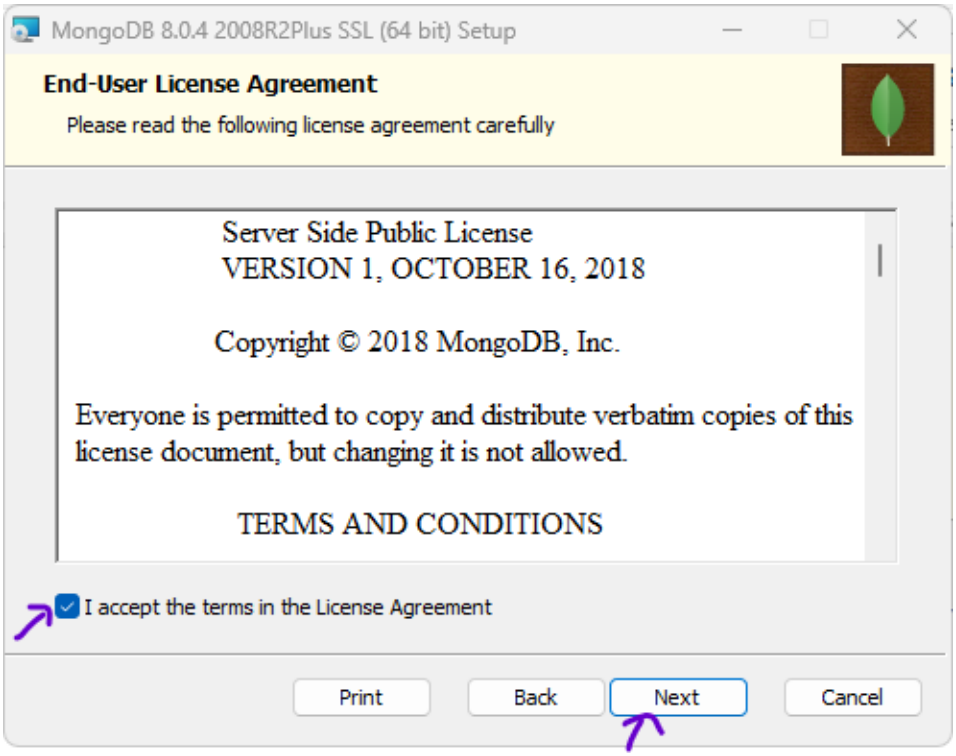




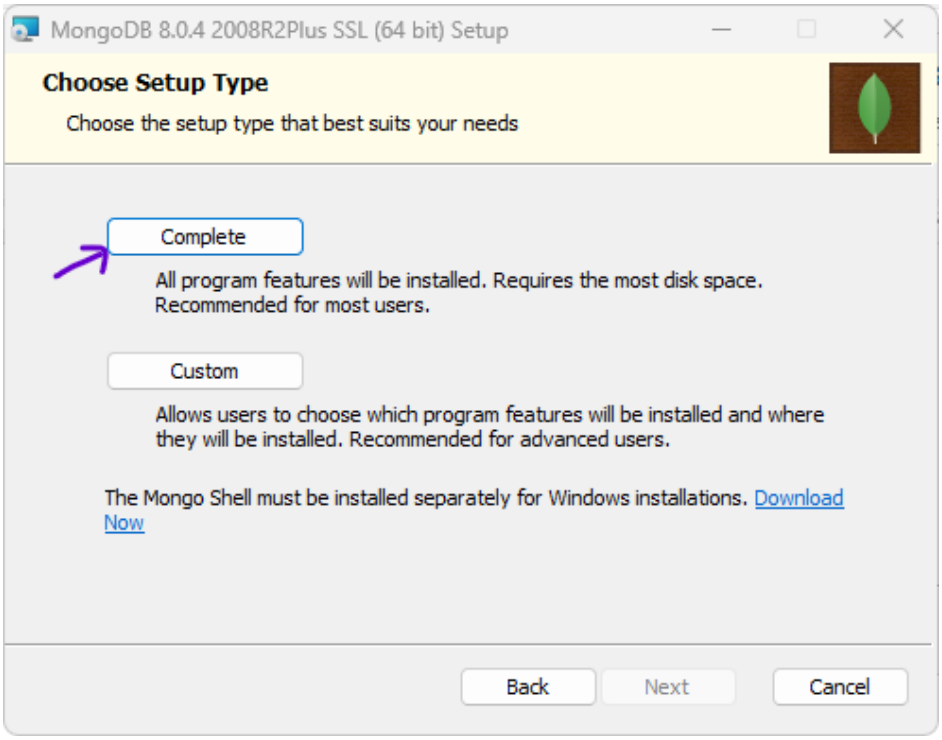
Double click to the file mongodb-windows-x86_64-8.0.4-signed to install it.

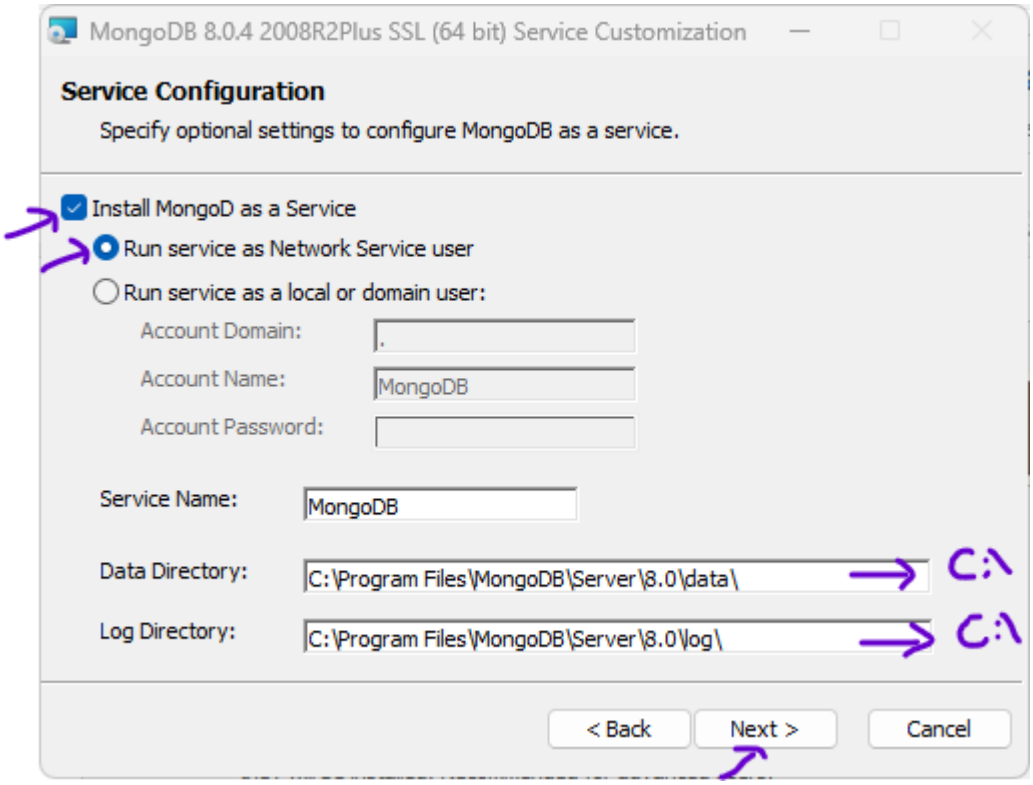


Accept the license.



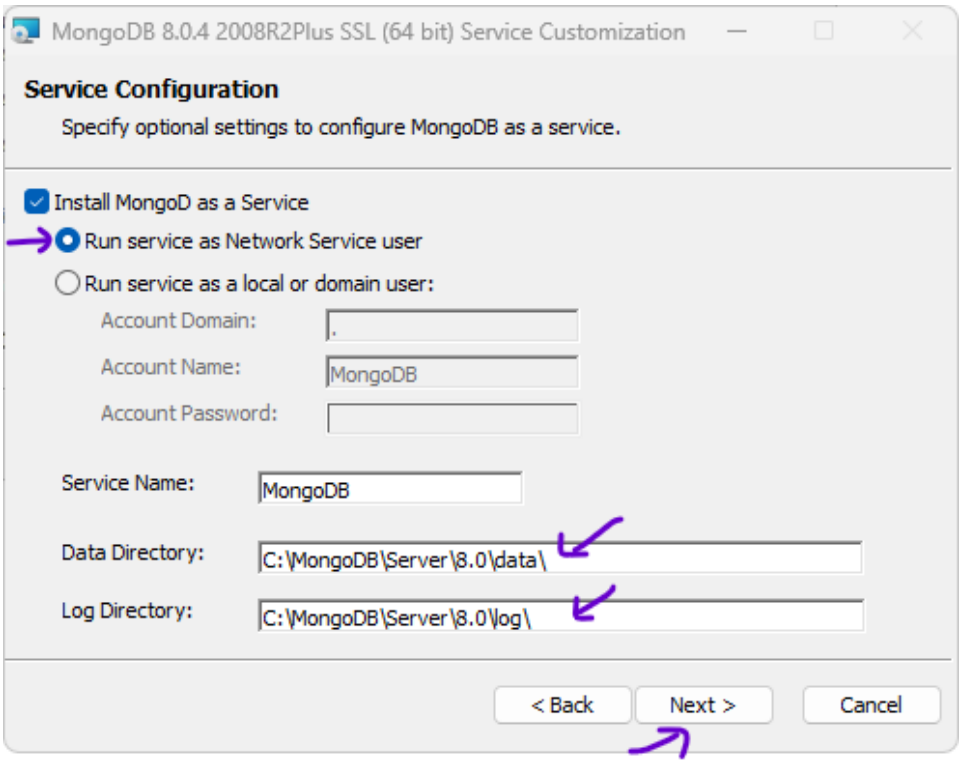
Click Complete.



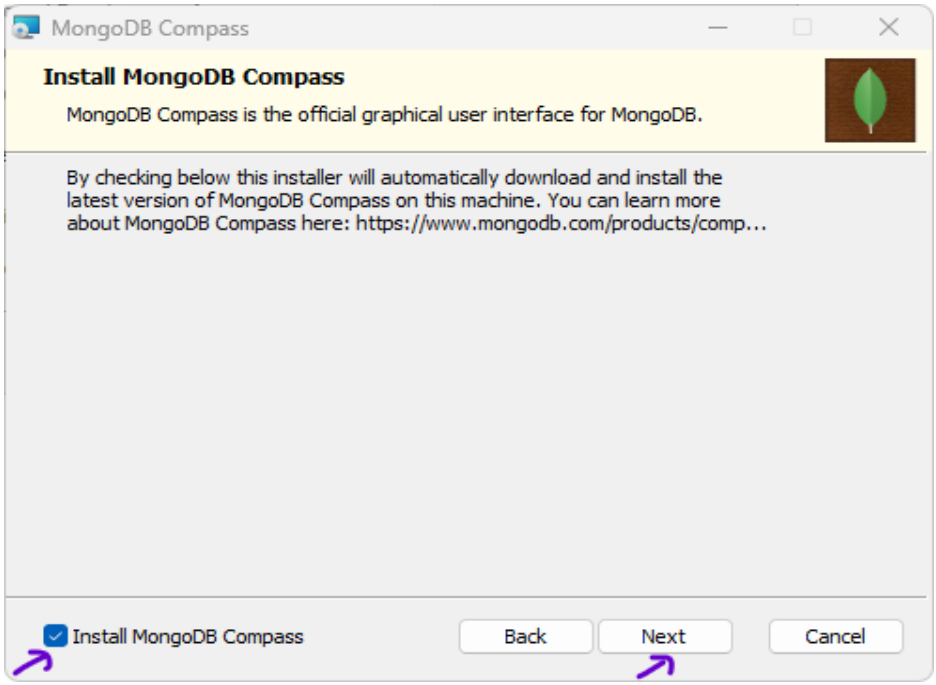


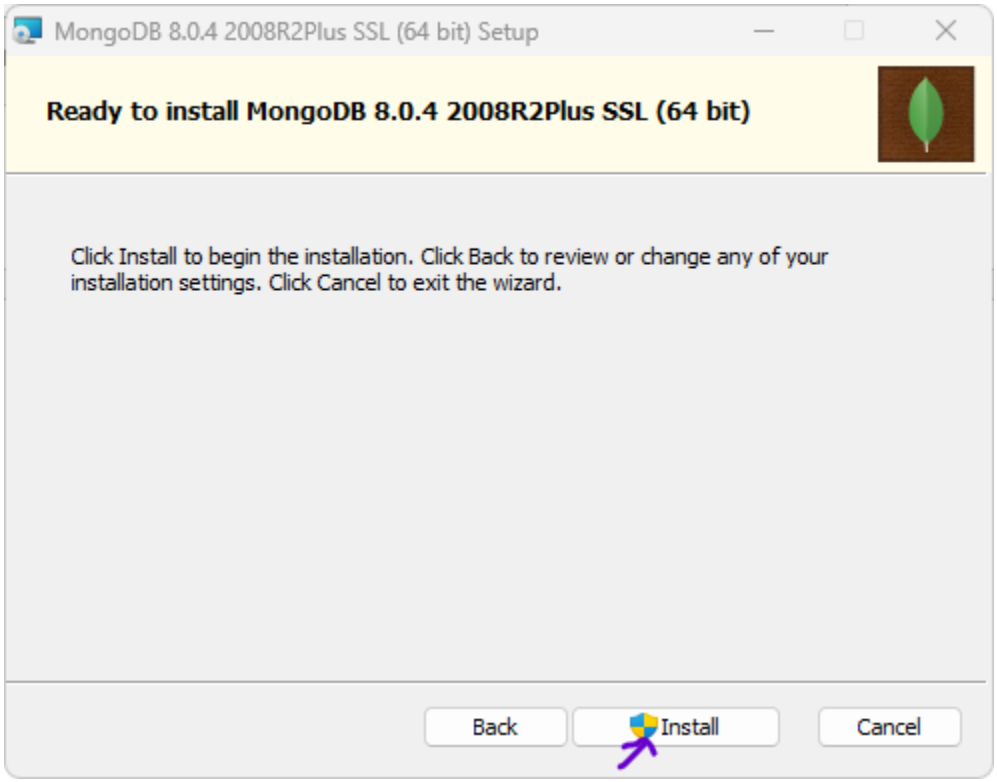
Choose:

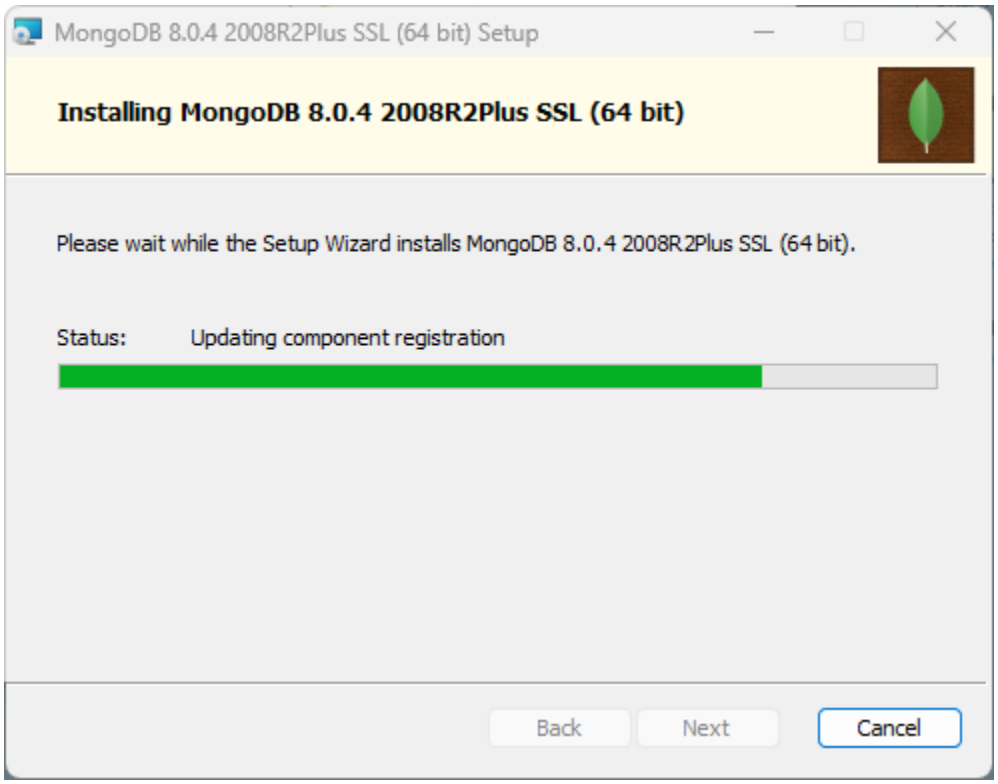
- Run service as Network Service user
- Install into C:\

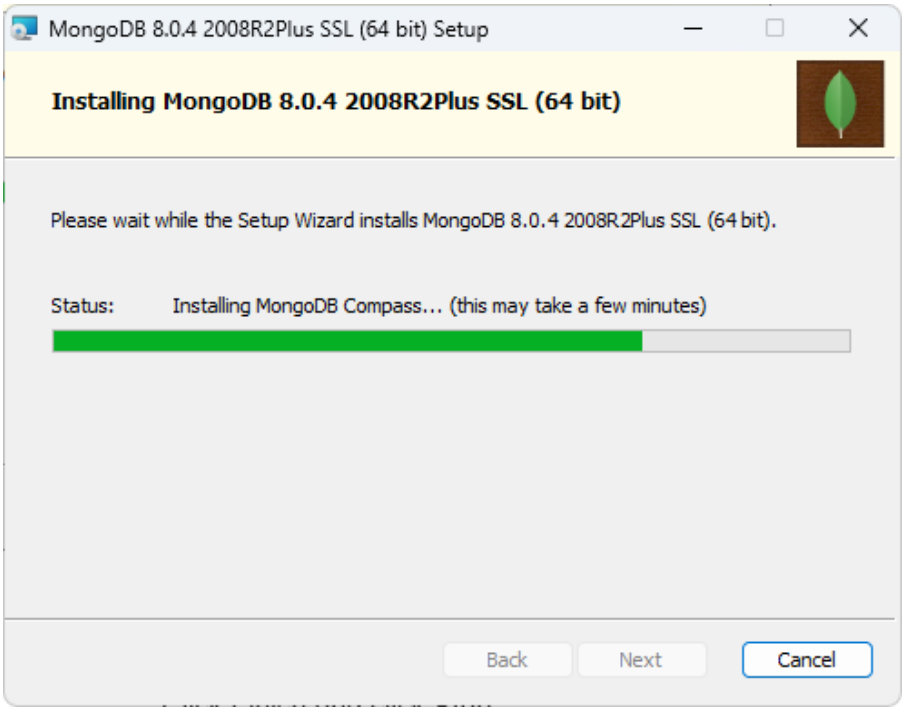


Install MongoDB Compass on your personal computer to execute database using UI (User Interface). Click Next. Click Install.

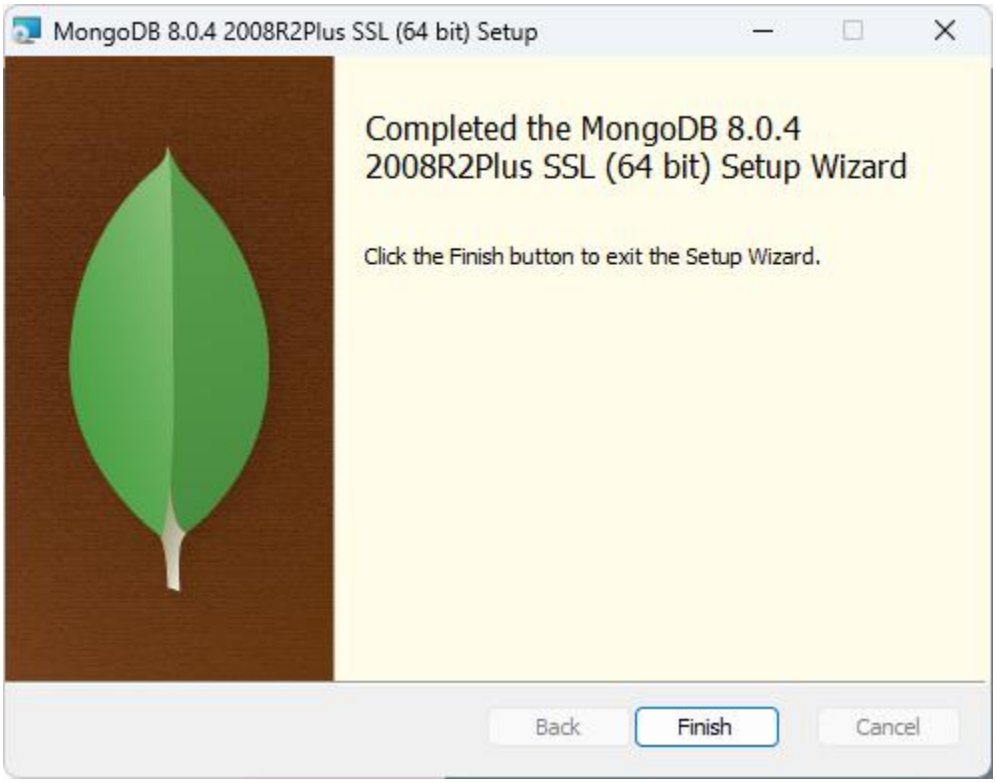


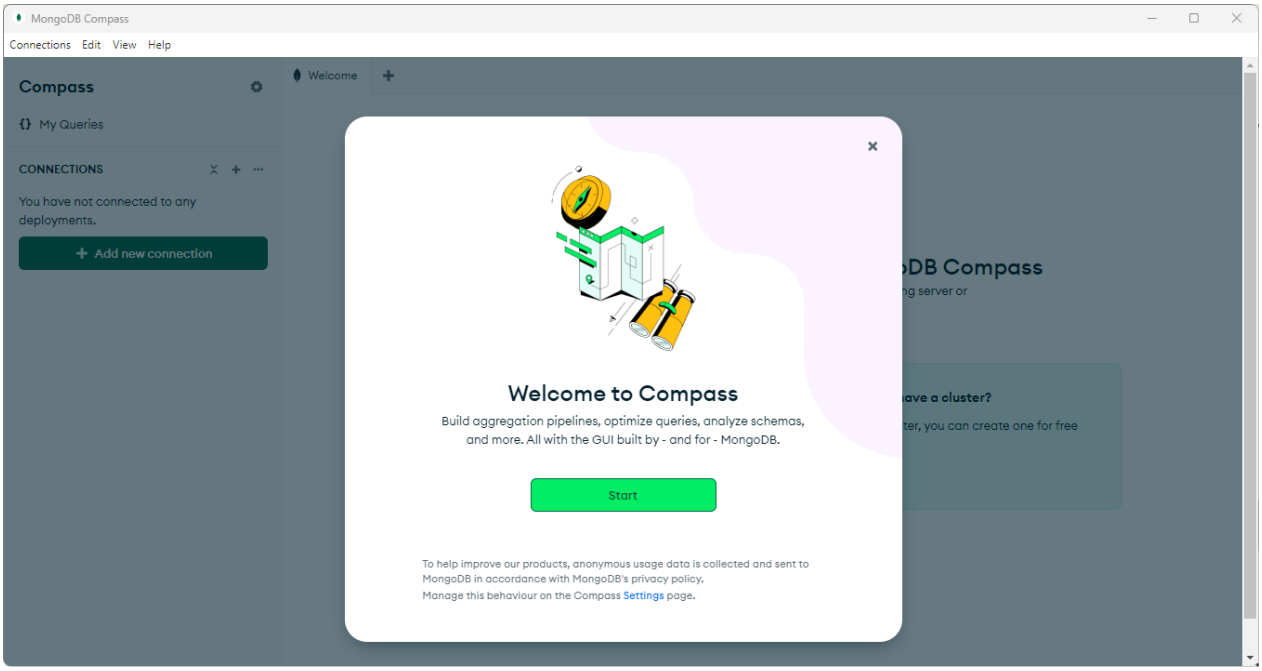

MongoDB 8.0.4 2008R2Plus SSL (64 bit) Setup
Ready to install MongoDB 8.0.4 2008R2Plus SSL (64 bit)
Click Install to begin the installation. Click Back to review or change any of your installation settings. Click Cancel to exit the wizard.
Back
Install
Cancel

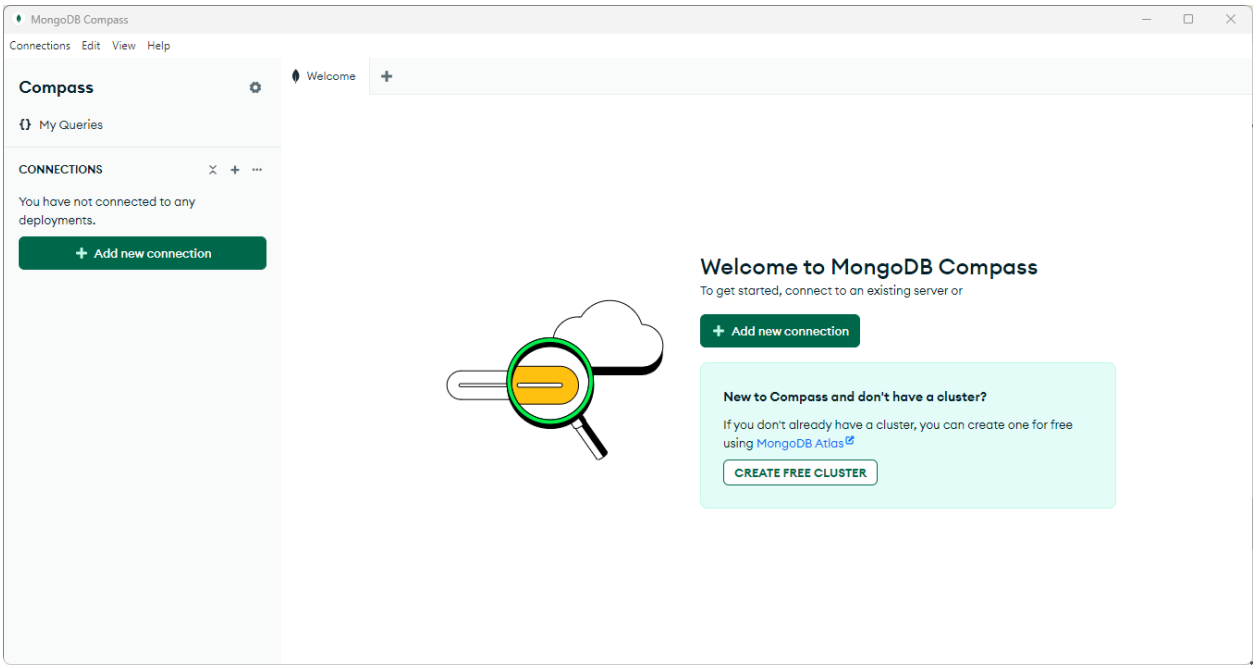

MongoDB 8.0.4 2008R2Plus SSL (64 bit) Setup
Installing MongoDB 8.0.4 2008R2Plus SSL (64 bit)
Please wait while the Setup Wizard installs MongoDB 8.0.4 2008R2Plus SSL (64 bit).
Status: Updating component registration
Back
Next
Cancel



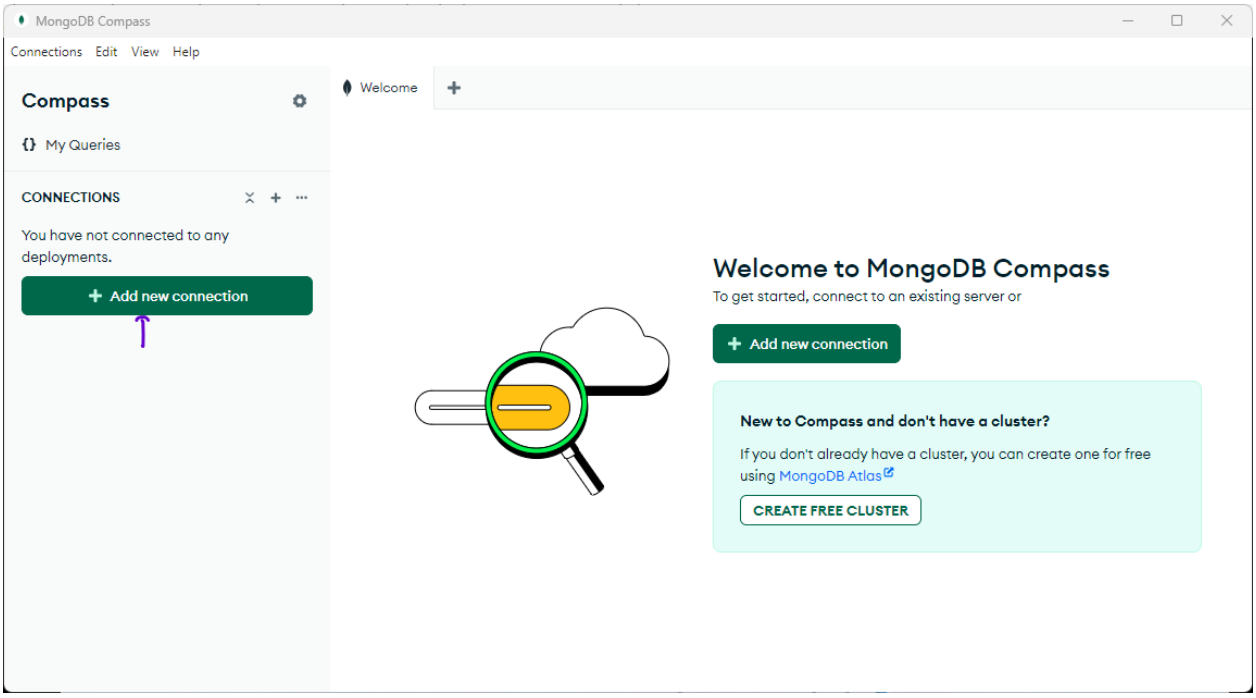
Click Finish and click Start to start Compass.

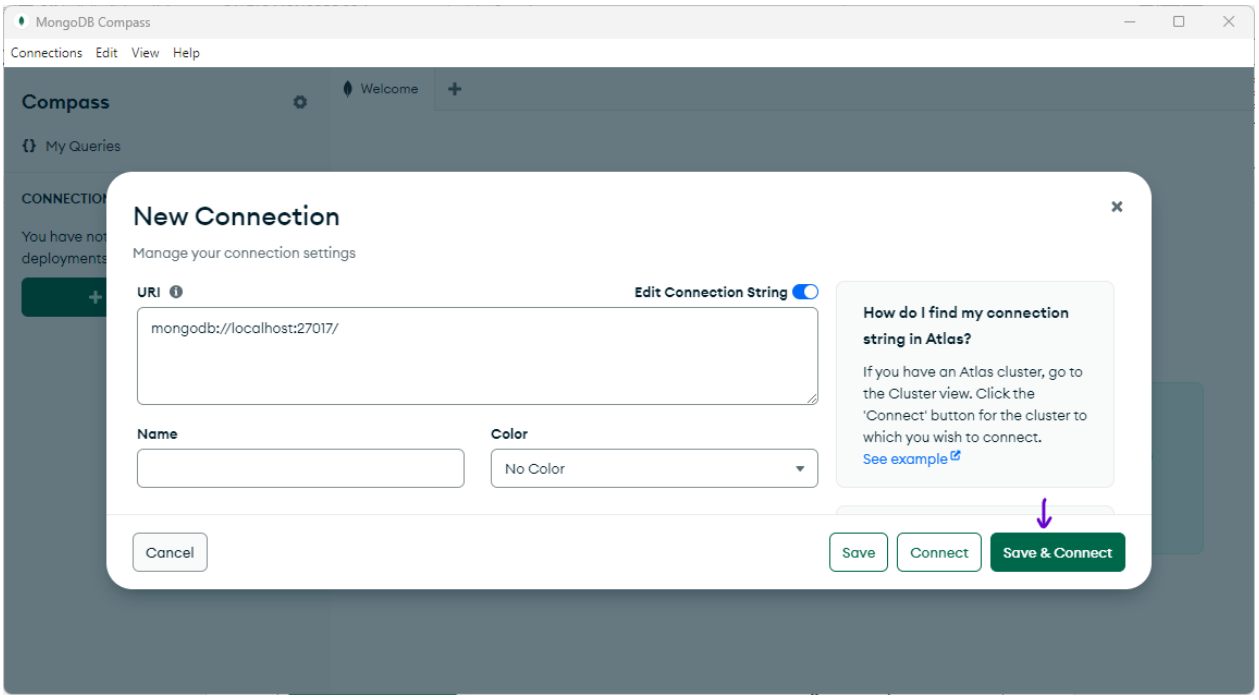


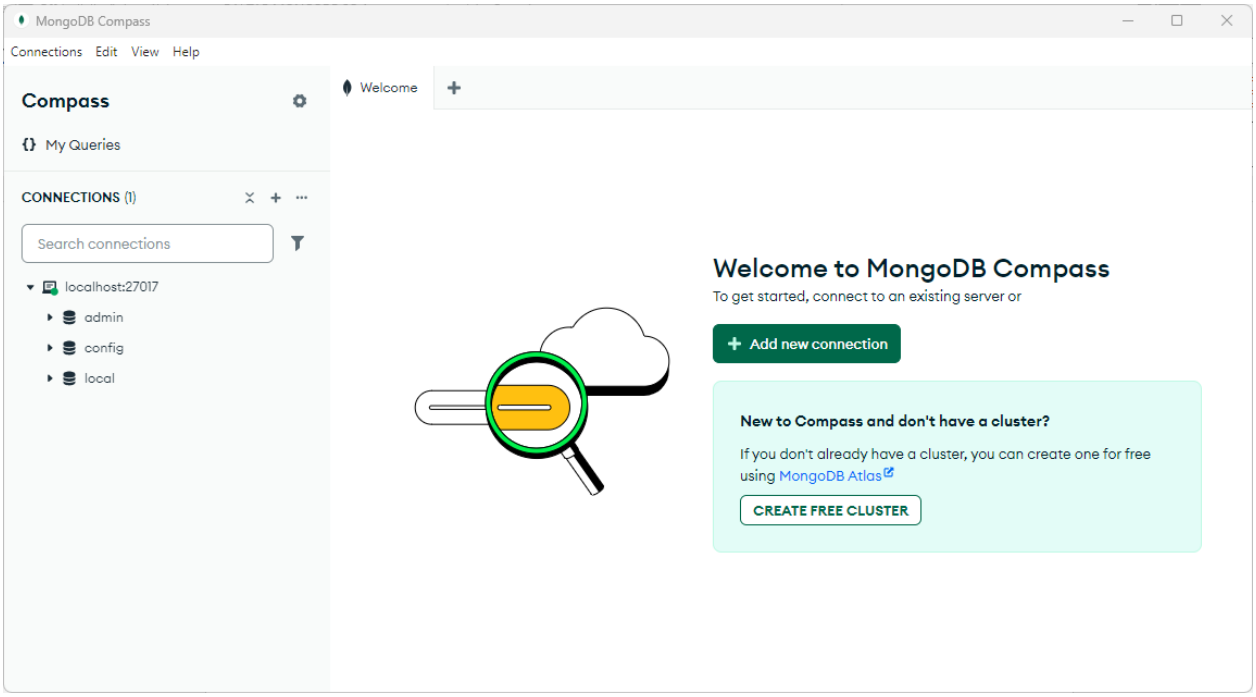


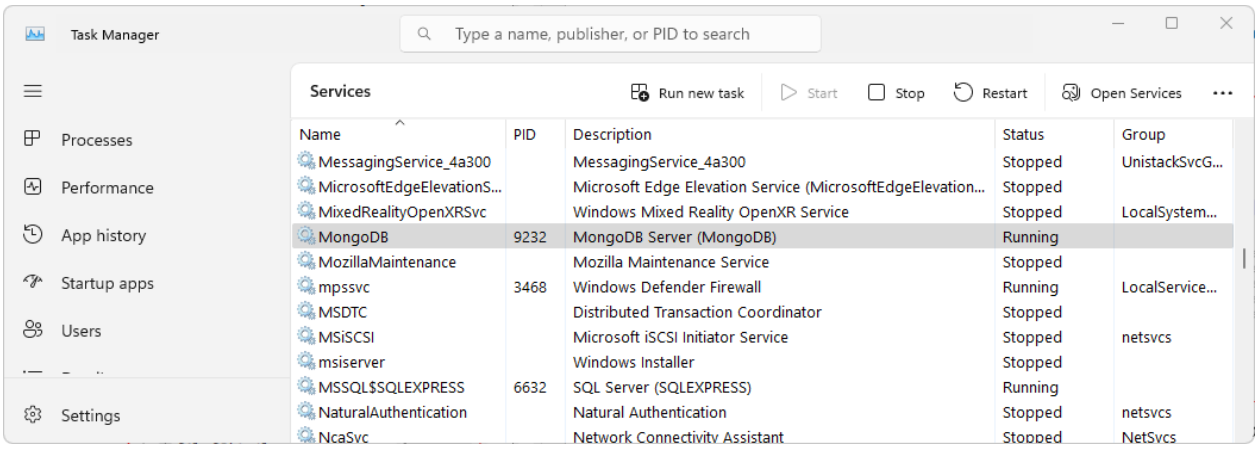


Click Add new connection and create a new connection by clicking Save & Connect. We can see all the databases given by MongoDB.


MongoDB Compass
Connections Edit View Help
Compass
My Queries
CONNECTIONS
You have not connected to any deployments.
Add new connection
Welcome
Welcome to MongoDB Compass
To get started, connect to an existing server or
Add new connection
New to Compass and don't have a cluster?
If you don't already have a cluster, you can create one for free using MongoDB Atlas
CREATE FREE CLUSTER

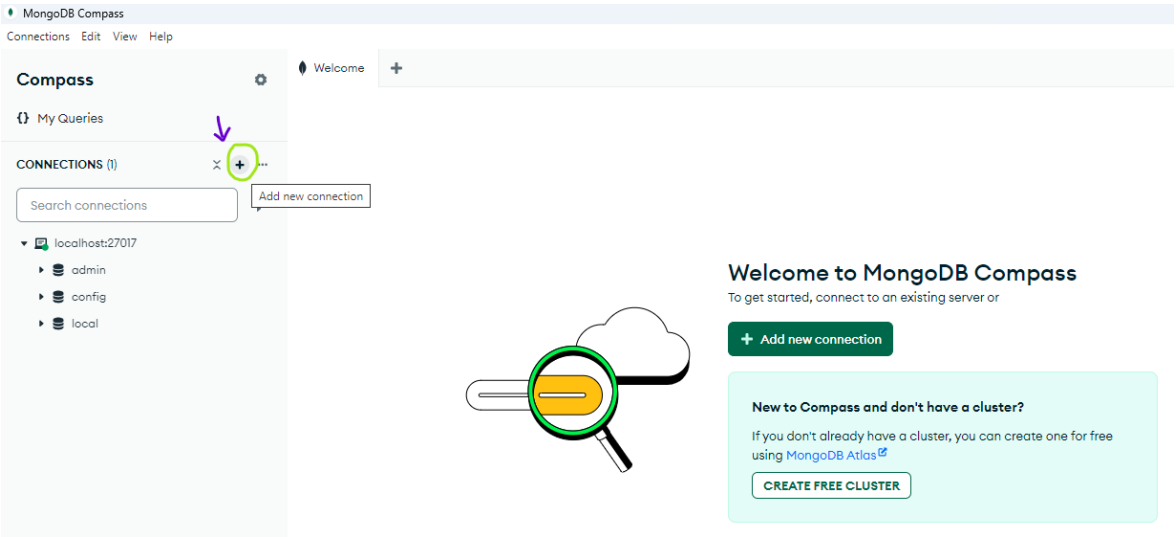

MongoDB Compass
Connections Edit View Help
Compass
My Queries
Welcome
New Connection
Manage your connection settings
URI
Edit Connection String
mongodb://localhost:27017/
Name
Color
No Color
How do I find my connection string in Atlas?
If you have an Atlas cluster, go to the Cluster view. Click the 'Connect' button for the cluster to which you wish to connect.
See example
Cancel
Save
Connect
Save & Connect

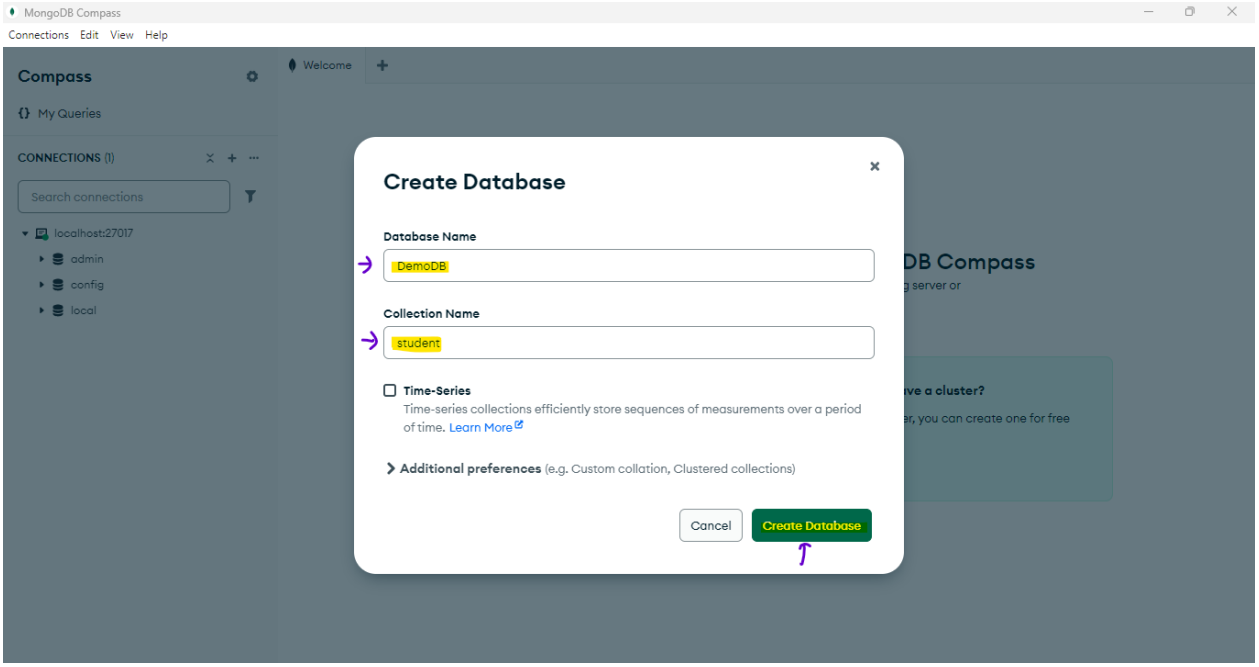


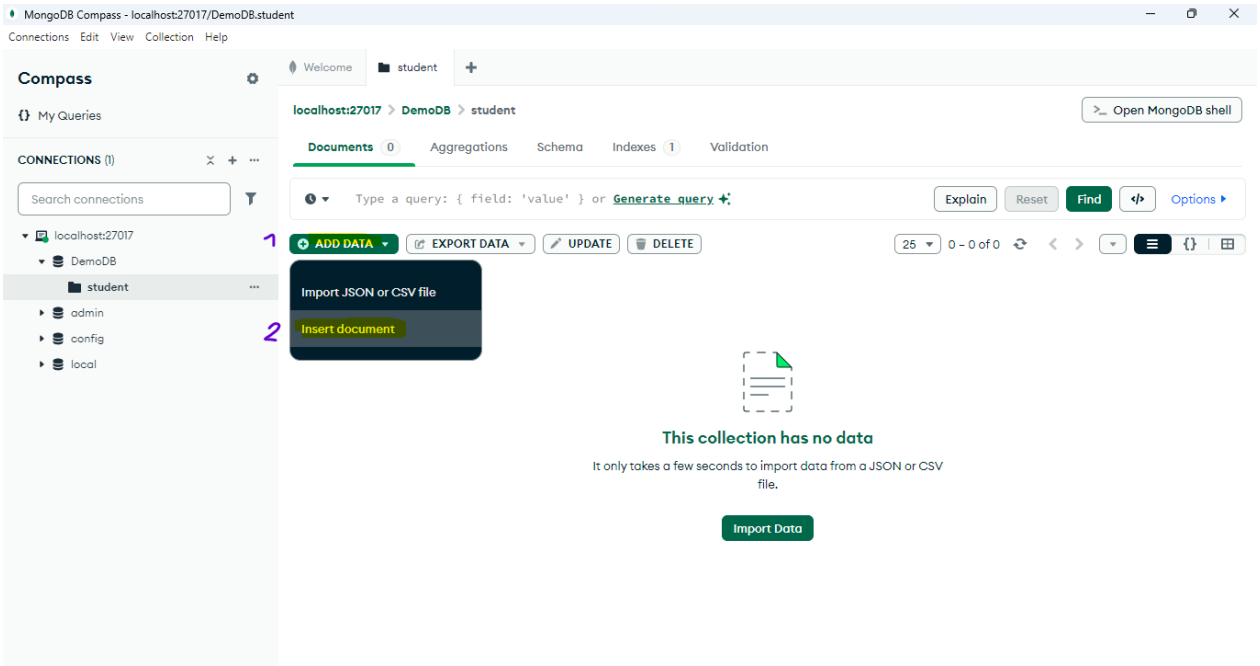


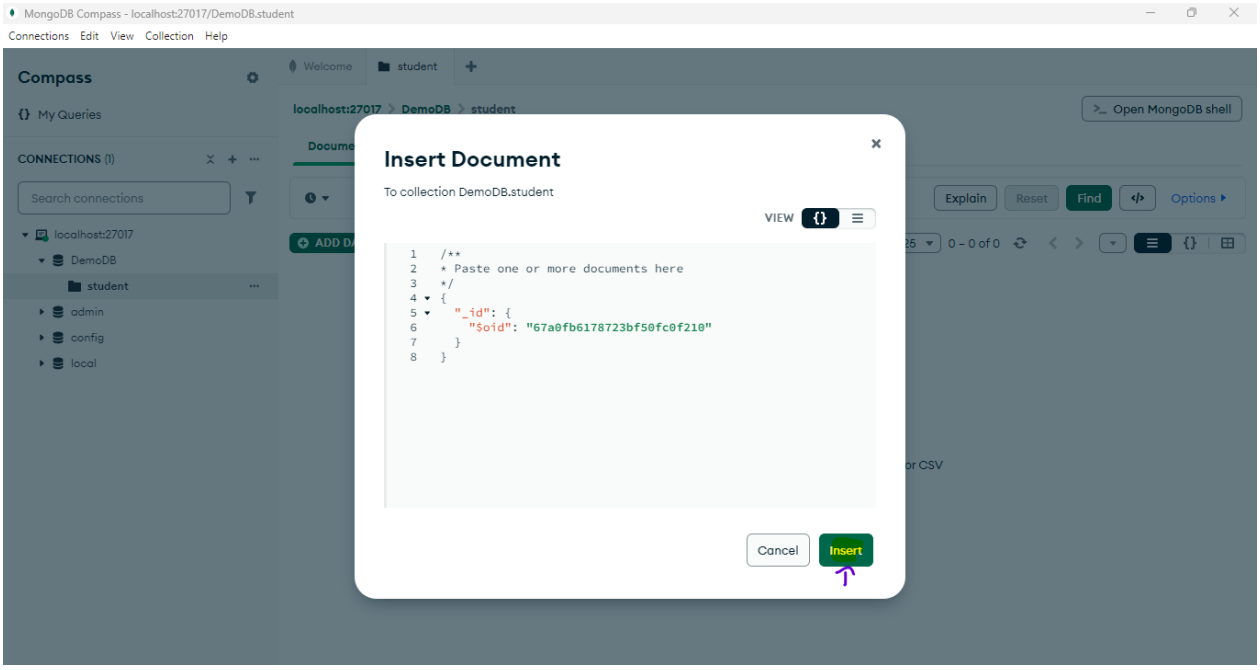
Try to create a new database

- Create a new database named DemoDB.
- Create a new collection named student into the database DemoDB.
- Add a new document int the collection student.


MongoDB Compass
Connections Edit View Help
Compass
My Queries
CONNECTIONS (1)
Search connections
Add new connection
localhost:27017
admin
config
local
Welcome
Welcome to MongoDB Compass
To get started, connect to an existing server or
Add new connection
New to Compass and don't have a cluster?
If you don't already have a cluster, you can create one for free using MongoDB Atlas
CREATE FREE CLUSTER


MongoDB Compass
Connections Edit View Help
Compass
My Queries
CONNECTIONS (1)
Search connections
localhost:27017
admin
config
local
Welcome
Create Database
Database Name
DemoDB
Collection Name
student
Time-Series
Time-series collections efficiently store sequences of measurements over a period of time. Learn More
Additional preferences (e.g. Custom collation, Clustered collections)
Cancel
Create Database



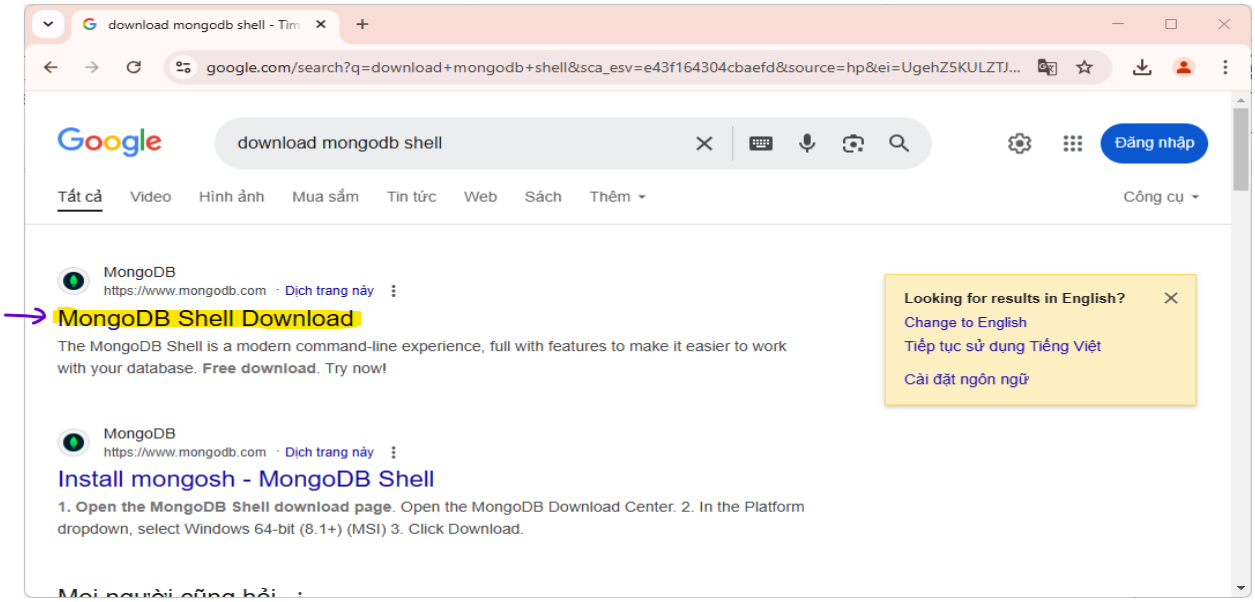


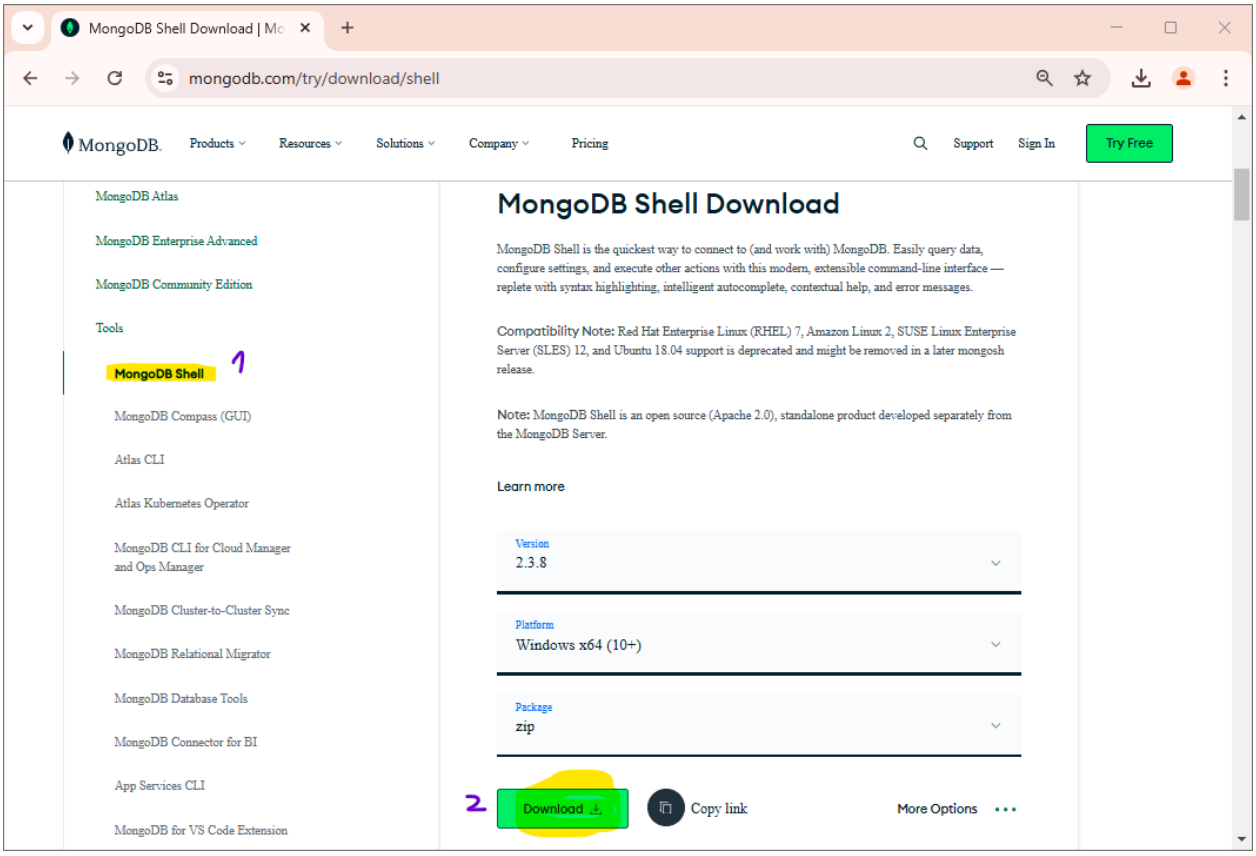
Click insert to insert a new document.

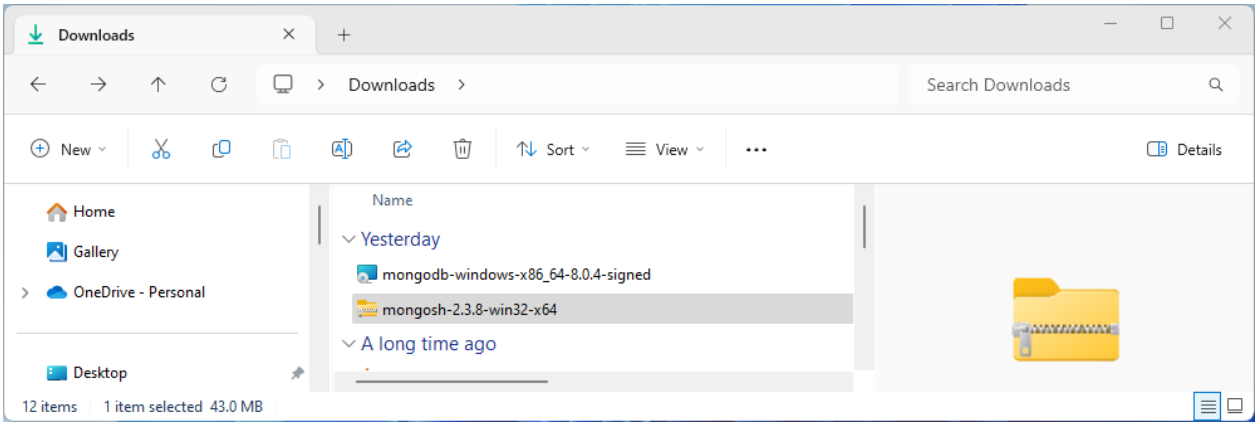
I.2. Install and Configure MongoDB Shell

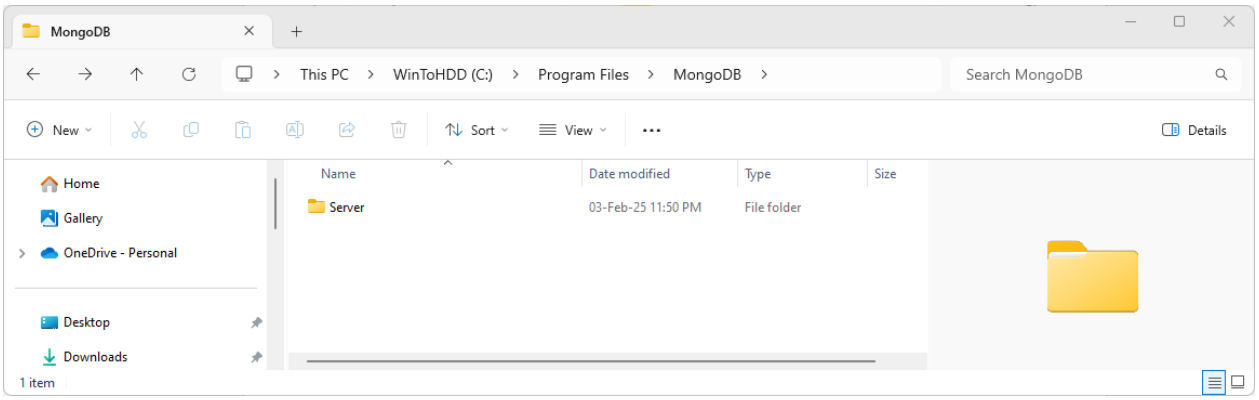
Download MongoDB shell at https://www.mongodb.com/try/download/shell.

Extract the mongosh-2.3.8-win32-x64.zip file to C:\Program Files\MongoDB.

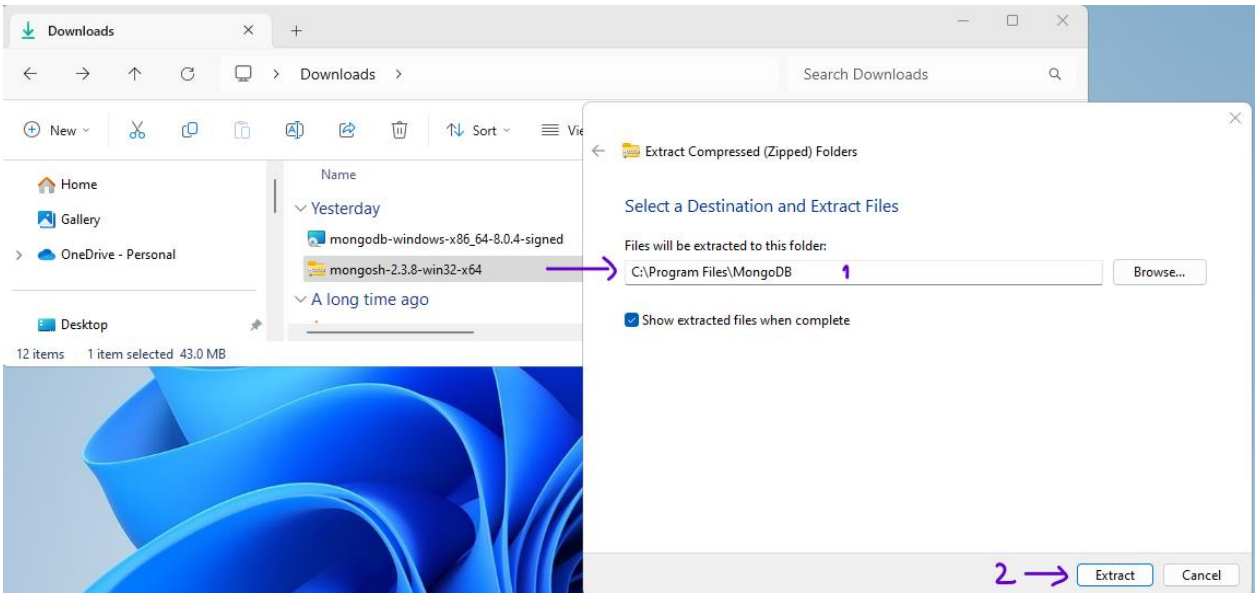


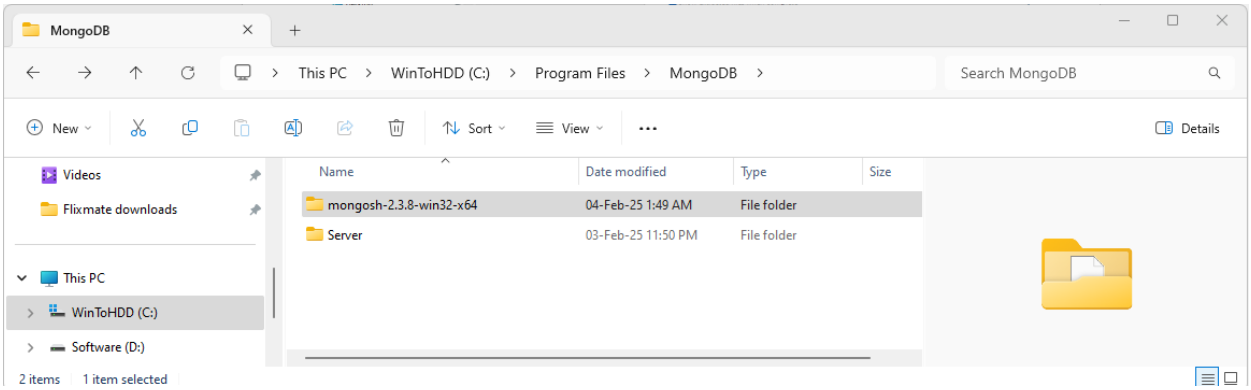






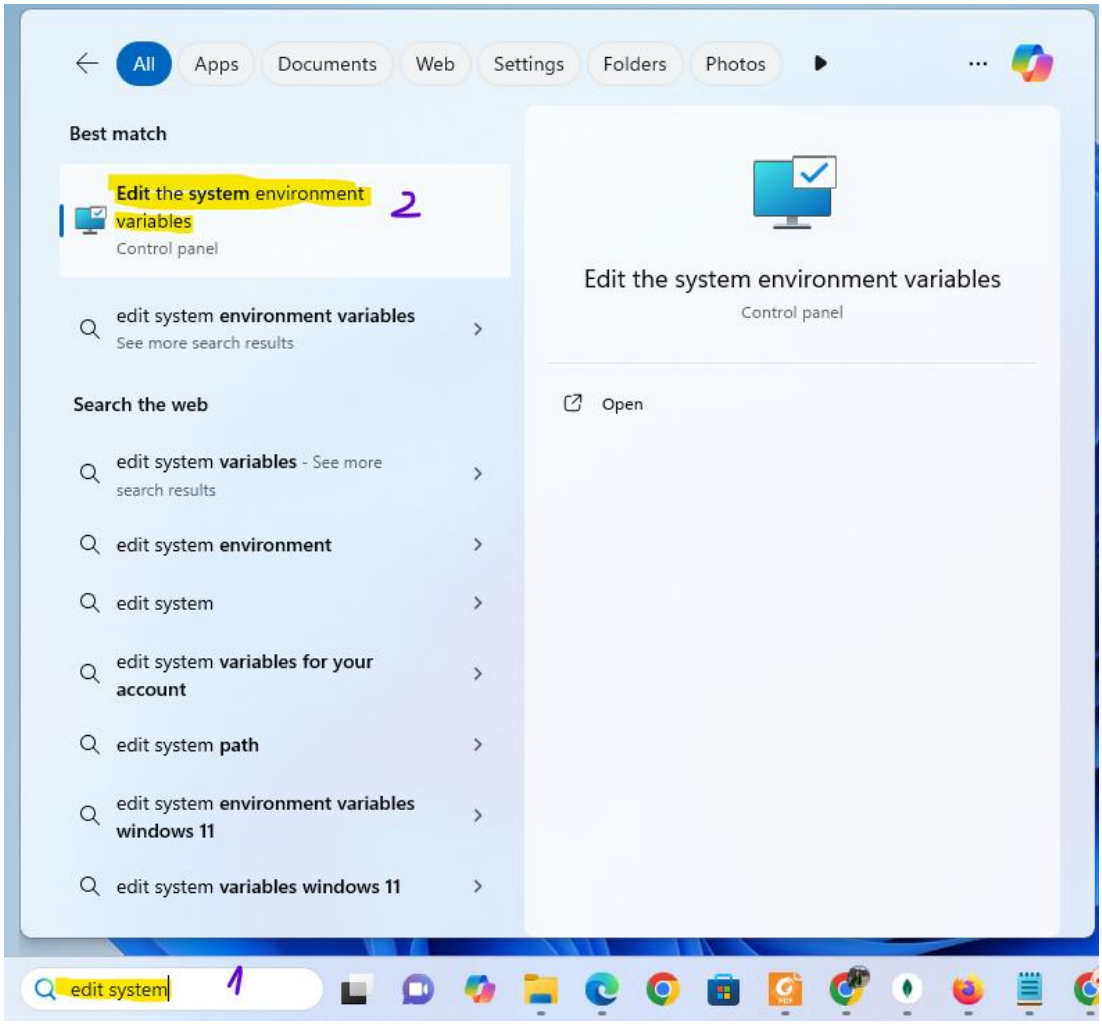
(The current C:\Program Files\MongoDB)

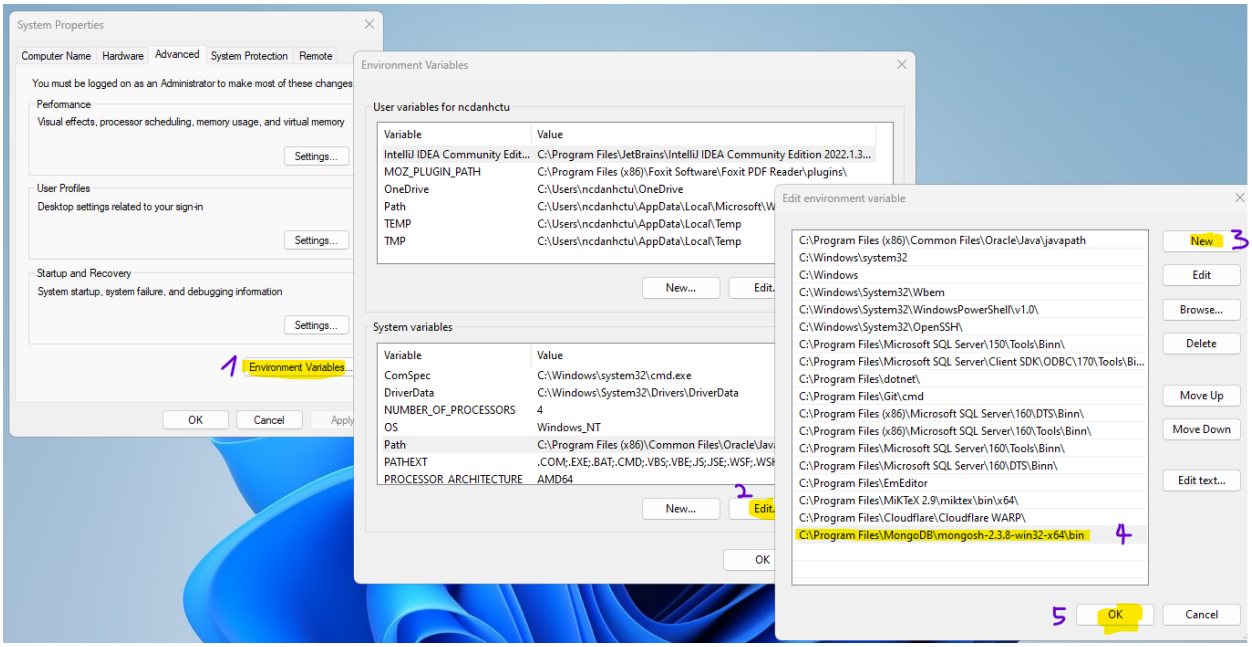




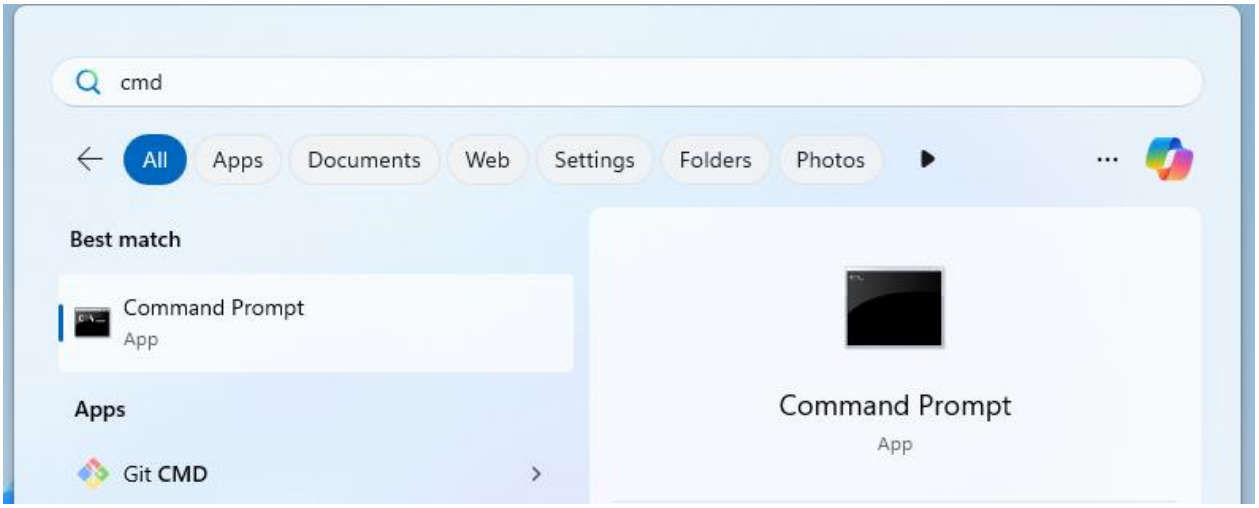
Set the environment variable

PATH = C:\Program Files\MongoDB\mongosh-2.3.8-win32-x64\bin





Open Command Prompt and try the MongoDB Shell.



Type:

> mongosh

> show db;

```
mongosh mongodb://127.0.0.

Microsoft Windows [Version 10.0.22621.4317]
(c) Microsoft Corporation. All rights reserved.

1→ C:\Users\ncdanhctu>mongosh
Current Mongosh Log ID: 67a11750e4b50639cdcb0ce1
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2
.3.8
Using MongoDB:          8.0.4
Using Mongosh:          2.3.8

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.co
m/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2025-02-03T23:57:48.049+07:00: Access control is not enabled for the database. Read and write access to data and conf
iguration is unrestricted
------

2 → test> show dbs;
DemoDB  40.00 KiB
admin   40.00 KiB
config  72.00 KiB
local   40.00 KiB
test>
```

II. MONGODB CONFIGURATION

1. Download from http://www.mongodb.org/downloads
2. Install **.msi** file in folder **C:\mongodb**
3. Create **data**, **data\db**, **log** directories and **mongo.config** file under **C:\mongodb**.
4. Add the following lines in "**mongo.config**" file

   port=27017

   dbpath=C:\mongodb\data\db\

   logpath=C:\mongodb\log\mongo.log
5. Start server :

   mongod.exe --config="C:\mongodb\mongo.config"
6. Connect to localhost MongoDB server via command line

   mongo --port 27017
7. Connect to remote MongoDB server via command line with authentication.
8. Create MongoDB Service in Windows. First Open cmd with administrator

   mongod --port 27017 --dbpath "a mongodb storage actual path e.g: d:\mongo_storage\data" --logpath="a log path e.g: d:\mongo_storage\log\log.txt" --install --serviceName "MongoDB"
9. After that

   Start Service

   net start MongoDB

   Stop Service

   net stop MongoDB

(see more: https://stackoverflow.com/questions/20796714/how-do-i-start-mongo-db-from-windows)

III. MONGODB BASIC OPERATIONS

List all databases

```
>_MONGOSH
> show databases
< DemoDB   708.00 KiB
  admin     40.00 KiB
  config    72.00 KiB
  local     40.00 KiB
```

(show database/dbs)

Transfer permissions to a database (if not, create a new one)

```
> use DemoDB
< switched to db DemoDB
```

Display all collections in that database

```
> show collections
< restaurants
  student
```

Display help

```
> help
< Shell Help

  use                          Set current database
  show                         'show databases'/'show dbs': Print a list of all available databases. 'show collections'/'show
                               tables': Print a list of all collections for current database. 'show profile': Prints
                               system.profile information. 'show users': Print a list of all users for current database. 'show
                               roles': Print a list of all roles for current database. 'show log <type>': log for current
                               connection, if type is not set uses 'global' 'show logs': Print all logs.
  exit                         Quit the MongoDB shell with exit/exit()/.exit
  quit                         Quit the MongoDB shell with quit/quit()
  Mongo                        Create a new connection and return the Mongo object. Usage: new Mongo(URI, options [optional])
  connect                      Create a new connection and return the Database object. Usage: connect(URI, username [optional],
                               password [optional])
  it                           result of the last line evaluated; use to further iterate
  version                      Shell version
  load                         Loads and runs a JavaScript file into the current shell environment
  enableTelemetry              Enables collection of anonymous usage data to improve the mongosh CLI
  disableTelemetry             Disables collection of anonymous usage data to improve the mongosh CLI
  passwordPrompt               Prompts the user for a password
  sleep                        Sleep for the specified number of milliseconds
  print                        Prints the contents of an object to the output
  printjson                    Alias for print()
  convertShardKeyToHashed      Returns the hashed value for the input using the same hashing function as a hashed index.
  cls                          Clears the screen like console.clear()
```

IV. CREATING A MONGODB DATABASE WITH THE CLI

MongoDB **use DATABASE_NAME** is used to create database. The command will create a new database if it doesn't exist, otherwise it will return the existing database.

Syntax

Basic syntax of **use DATABASE** statement is as follows −

```
use DATABASE_NAME
```

Example

If you want to use a database with name <mydb>, then use DATABASE statement would be as follows −

```
>use mydb

switched to db mydb
```

To check your currently selected database, use the command **db**

```
>db

mydb
```

If you want to check your databases list, use the command **show dbs**.

```
>show dbs
local     0.78125GB
test      0.23012GB
```

Your created database (mydb) is not present in list. To display database, you need to insert at least one document into it.

```
>db.movie.insert({"name":"tutorials point"})
>show dbs
local      0.78125GB
mydb       0.23012GB
test       0.23012GB
```

In MongoDB default database is test. If you didn't create any database, then collections will be stored in test database.

V. MONGODB CRUD OPERATIONS

(see: https://www.mongodb.com/docs/manual/crud/)

CRUD operations *create*, *read*, *update*, and *delete* documents.

You can connect with driver methods and perform CRUD operations for deployments hosted in the following environments:

You can perform CRUD operations in the UI for deployments hosted in MongoDB Atlas.

Create Operations

Create or insert operations add new documents to a collection. If the collection does not currently exist, insert operations will create the collection.

MongoDB provides the following methods to insert documents into a collection:

- db.collection.insertOne()
- db.collection.insertMany()

In MongoDB, insert operations target a single collection. All write operations in MongoDB are atomic on the level of a single document.

```
db.users.insertOne(          ← collection
   {
      name: "sue",           ← field: value
      age: 26,               ← field: value   } document
      status: "pending"      ← field: value
   }
)
```

For examples, see Insert Documents.

Read Operations

Read operations retrieve documents from a collection; i.e. query a collection for documents. MongoDB provides the following methods to read documents from a collection:

- db.collection.find()

You can specify query filters or criteria that identify the documents to return.

```
db.users.find(                          ← collection
   { age: { $gt: 18 } },                ← query criteria
   { name: 1, address: 1 }              ← projection
).limit(5)                              ← cursor modifier
```

For examples, see:

- Query Documents
- Query on Embedded/Nested Documents
- Query an Array
- Query an Array of Embedded Documents

Update Operations

Update operations modify existing documents in a collection. MongoDB provides the following methods to update documents of a collection:

- db.collection.updateOne()
- db.collection.updateMany()
- db.collection.replaceOne()

In MongoDB, update operations target a single collection. All write operations in MongoDB are atomic on the level of a single document.

You can specify criteria, or filters, that identify the documents to update. These filters use the same syntax as read operations.

```
db.users.updateMany(                    ← collection
   { age: { $lt: 18 } },                ← update filter
   { $set: { status: "reject" } }       ← update action
)
```

For examples, see Update Documents.

Delete Operations

Delete operations remove documents from a collection. MongoDB provides the following methods to delete documents of a collection:

- db.collection.deleteOne()
- db.collection.deleteMany()

In MongoDB, delete operations target a single collection. All write operations in MongoDB are atomic on the level of a single document.

You can specify criteria, or filters, that identify the documents to remove. These filters use the same syntax as read operations.

```
db.users.deleteMany(        ← collection
   { status: "reject" }     ← delete filter
)
```

For examples, see Delete Documents.

Bulk Write

MongoDB provides the ability to perform write operations in bulk. For details, see Bulk Write Operations.

Import Data

Import dữ liệu file json (zips.json) vào mongo

mongoimport –host localhost --port 27017 --db sampledb - -collection zips --file zips.json

SQL server vs. MongoDB

SQL Terms/Concept	MongoDB Terms/Concepts
database	database
table	collection
row	document or BSON document
column	field
index	index
table joins	$lookup, embedded documents
primary key Specify any unique column or column combination as primary key	primary key In MongoDB, the primary key is automatically set to the _id field
aggregation (e.g. group by)	aggregation pipeline See the SQL to Aggregation Mapping Chart.

Read more at: https://docs.mongodb.com/manual/reference/sql-comparison/

VI. MONGODB QUERY EXERCISES

- restaurants collection [87 Exercises]
- Movies collection [28 Exercises]

VI.1. MongoDB Exercises: restaurants collection

For restaurant documents stored at:

https://www.w3resource.com/mongodb-exercises/retaurants.zip

Structure of 'restaurants' collection:

```
{
  "address": {
     "building": "1007",
     "coord": [ -73.856077, 40.848447 ],
     "street": "Morris Park Ave",
     "zipcode": "10462"
  },
  "borough": "Bronx",
  "cuisine": "Bakery",
  "grades": [
     { "date": { "$date": 1393804800000 }, "grade": "A", "score": 2 },
     { "date": { "$date": 1378857600000 }, "grade": "A", "score": 6 },
     { "date": { "$date": 1358985600000 }, "grade": "A", "score": 10 },
     { "date": { "$date": 1322006400000 }, "grade": "A", "score": 9 },
     { "date": { "$date": 1299715200000 }, "grade": "B", "score": 14 }
  ],
  "name": "Morris Park Bake Shop",
  "restaurant_id": "30075445"
}
```

(https://www.w3resource.com/mongodb-exercises/)

You may download the compressed file and uncompress it to find the collection used in our exercises. The collection comprises of 3772 documents. (Download the document, import it into mongodb).

1. Write a MongoDB query to display all the documents in the collection restaurants.

2. Write a MongoDB query to display the fields restaurant_id, name, borough and cuisine for all the documents in the collection restaurant.

3. Write a MongoDB query to display the fields restaurant_id, name, borough and cuisine, but exclude the field _id for all the documents in the collection restaurant.

4. Write a MongoDB query to display the fields restaurant_id, name, borough and zip code, but exclude the field _id for all the documents in the collection restaurant.

5. Write a MongoDB query to display all the restaurants which is in the borough Bronx.

6. Write a MongoDB query to display the first 5 restaurants which is in the borough Bronx.

7.Write a MongoDB query to display the next 5 restaurants after skipping first 5 which are in the borough Bronx.

8. Write a MongoDB query to find the restaurants who achieved a score more than 90.

9. Write a MongoDB query to find the restaurants that achieved a score, more than 80 but less than 100.

10. Write a MongoDB query to find the restaurants which locate in latitude value less than -95.754168.

11. Write a MongoDB query to find the restaurants that do not prepare any cuisine of 'American' and their grade score more than 70 and latitude less than -65.754168.

12. Write a MongoDB query to find the restaurants which do not prepare any cuisine of 'American' and achieved a score more than 70 and located in the longitude less than -65.754168.
Note : Do this query without using $and operator.

13. Write a MongoDB query to find the restaurants which do not prepare any cuisine of 'American' and achieved a grade point 'A' not belongs to the borough Brooklyn. The document must be displayed according to the cuisine in descending order.

14. Write a MongoDB query to find the restaurant Id, name, borough and cuisine for those restaurants which contain 'Wil' as first three letters for its name.

15. Write a MongoDB query to find the restaurant Id, name, borough and cuisine for those restaurants which contain 'ces' as last three letters for its name.

16. Write a MongoDB query to find the restaurant Id, name, borough and cuisine for those restaurants which contain 'Reg' as three letters somewhere in its name.

17. Write a MongoDB query to find the restaurants which belong to the borough Bronx and prepared either American or Chinese dish.

18. Write a MongoDB query to find the restaurant Id, name, borough and cuisine for those restaurants which belong to the borough Staten Island or Queens or Bronxor Brooklyn.

19. Write a MongoDB query to find the restaurant Id, name, borough and cuisine for those restaurants which are not belonging to the borough Staten Island or Queens or Bronxor Brooklyn.

20. Write a MongoDB query to find the restaurant Id, name, borough and cuisine for those restaurants which achieved a score which is not more than 10.

21. Write a MongoDB query to find the restaurant Id, name, borough and cuisine for those restaurants which prepared dish except 'American' and 'Chinese' or restaurant's name begins with letter 'Wil'.

22. Write a MongoDB query to find the restaurant Id, name, and grades for those restaurants which achieved a grade of "A" and scored 11 on an ISODate "2014-08-11T00:00:00Z" among many of survey dates..
23. Write a MongoDB query to find the restaurant Id, name and grades for those restaurants where the 2nd element of grades array contains a grade of "A" and score 9 on an ISODate "2014-08-11T00:00:00Z".

24. Write a MongoDB query to find the restaurant Id, name, address and geographical location for those restaurants where 2nd element of coord array contains a value which is more than 42 and upto 52.

25. Write a MongoDB query to arrange the name of the restaurants in ascending order along with all the columns.

26. Write a MongoDB query to arrange the name of the restaurants in descending along with all the columns.

27. Write a MongoDB query to arranged the name of the cuisine in ascending order and for that same cuisine borough should be in descending order.

28. Write a MongoDB query to know whether all the addresses contains the street or not.

29. Write a MongoDB query which will select all documents in the restaurants collection where the coord field value is Double.

30. Write a MongoDB query which will select the restaurant Id, name and grades for those restaurants which returns 0 as a remainder after dividing the score by 7.

31. Write a MongoDB query to find the restaurant name, borough, longitude and attitude and cuisine for those restaurants which contains 'mon' as three letters somewhere in its name.

32. Write a MongoDB query to find the restaurant name, borough, longitude and latitude and cuisine for those restaurants which contain 'Mad' as first three letters of its name.

33. Write a MongoDB query to find the restaurants that have at least one grade with a score of less than 5.

34. Write a MongoDB query to find the restaurants that have at least one grade with a score of less than 5 and that are located in the borough of Manhattan.

35. Write a MongoDB query to find the restaurants that have at least one grade with a score of less than 5 and that are located in the borough of Manhattan or Brooklyn.

36. Write a MongoDB query to find the restaurants that have at least one grade with a score of less than 5 and that are located in the borough of Manhattan or Brooklyn, and their cuisine is not American.

37. Write a MongoDB query to find the restaurants that have at least one grade with a score of less than 5 and that are located in the borough of Manhattan or Brooklyn, and their cuisine is not American or Chinese.

38. Write a MongoDB query to find the restaurants that have a grade with a score of 2 and a grade with a score of 6.

39. Write a MongoDB query to find the restaurants that have a grade with a score of 2 and a grade with a score of 6 and are located in the borough of Manhattan.

40. Write a MongoDB query to find the restaurants that have a grade with a score of 2 and a grade with a score of 6 and are located in the borough of Manhattan or Brooklyn.

41. Write a MongoDB query to find the restaurants that have a grade with a score of 2 and a grade with a score of 6 and are located in the borough of Manhattan or Brooklyn, and their cuisine is not American.

42. Write a MongoDB query to find the restaurants that have a grade with a score of 2 and a grade with a score of 6 and are located in the borough of Manhattan or Brooklyn, and their cuisine is not American or Chinese.

43. Write a MongoDB query to find the restaurants that have a grade with a score of 2 or a grade with a score of 6.

44. Write a MongoDB query to find the restaurants that have a grade with a score of 2 or a grade with a score of 6 and are located in the borough of Manhattan.

45. Write a MongoDB query to find the restaurants that have a grade with a score of 2 or a grade with a score of 6 and are located in the borough of Manhattan or Brooklyn.

46. Write a MongoDB query to find the restaurants that have a grade with a score of 2 or a grade with a score of 6 and are located in the borough of Manhattan or Brooklyn, and their cuisine is not American.

47. Write a MongoDB query to find the restaurants that have a grade with a score of 2 or a grade with a score of 6 and are located in the borough of Manhattan or Brooklyn, and their cuisine is not American or Chinese.

48. Write a MongoDB query to find the restaurants that have all grades with a score greater than 5.

49. Write a MongoDB query to find the restaurants that have all grades with a score greater than 5 and are located in the borough of Manhattan.

50. Write a MongoDB query to find the restaurants that have all grades with a score greater than 5 and are located in the borough of Manhattan or Brooklyn.

51. Write a MongoDB query to find the average score for each restaurant.

52. Write a MongoDB query to find the highest score for each restaurant.

53. Write a MongoDB query to find the lowest score for each restaurant.

54. Write a MongoDB query to find the count of restaurants in each borough.

55. Write a MongoDB query to find the count of restaurants for each cuisine.

56. Write a MongoDB query to find the count of restaurants for each cuisine and borough.

57. Write a MongoDB query to find the count of restaurants that received a grade of 'A' for each cuisine.

58. Write a MongoDB query to find the count of restaurants that received a grade of 'A' for each borough.

59. Write a MongoDB query to find the count of restaurants that received a grade of 'A' for each cuisine and borough.

60. Write a MongoDB query to find the number of restaurants that have been graded in each month of the year.

61. Write a MongoDB query to find the average score for each cuisine.

62. Write a MongoDB query to find the highest score for each cuisine.

63. Write a MongoDB query to find the lowest score for each cuisine.

64. Write a MongoDB query to find the average score for each borough.

65. Write a MongoDB query to find the highest score for each borough.

66. Write a MongoDB query to find the lowest score for each borough.

67. Write a MongoDB query to find the name and address of the restaurants that received a grade of 'A' on a specific date.

68. Write a MongoDB query to find the name and address of the restaurants that received a grade of 'B' or 'C' on a specific date.

69. Write a MongoDB query to find the name and address of the restaurants that have at least one 'A' grade and one 'B' grade.

70. Write a MongoDB query to find the name and address of the restaurants that have at least one 'A' grade and no 'B' grades.

71. Write a MongoDB query to find the name ,address and grades of the restaurants that have at least one 'A' grade and no 'C' grades.

72. Write a MongoDB query to find the name, address, and grades of the restaurants that have at least one 'A' grade, no 'B' grades, and no 'C' grades.

73. Write a MongoDB query to find the name and address of the restaurants that have the word 'coffee' in their name.

74. Write a MongoDB query to find the name and address of the restaurants that have a zipcode that starts with '10'.

75. Write a MongoDB query to find the name and address of the restaurants that have a cuisine that starts with the letter 'B'.

76. Write a MongoDB query to find the name, address, and cuisine of the restaurants that have a cuisine that ends with the letter 'y'.

77. Write a MongoDB query to find the name, address, and cuisine of the restaurants that have a cuisine that contains the word 'Pizza'.

78. Write a MongoDB query to find the restaurants achieved highest average score.

79. Write a MongoDB query to find all the restaurants with the highest number of "A" grades.

80. Write a MongoDB query to find the cuisine type that is most likely to receive a "C" grade.

81. Write a MongoDB query to find the restaurant that has the highest average score for thecuisine "Turkish".

82. Write a MongoDB query to find the restaurants that achieved the highest total score.

83. Write a MongoDB query to find all the Chinese restaurants in Brooklyn.

84. Write a MongoDB query to find the restaurant with the most recent grade date.

85. Write a MongoDB query to find the top 5 restaurants with the highest average score for each cuisine type, along with their average scores.

86. Write a MongoDB query to find the top 5 restaurants in each borough with the highest number of "A" grades.

87. Write a MongoDB query to find the borough with the highest number of restaurants that have a grade of "A" and a score greater than or equal to 90.

Sample Restaurants Dataset

The **sample_restaurants** database contains two collections specifically designed to help familiarize you with GeoJSON data.
To learn how to load the sample data provided by Atlas into your cluster, see Load Sample Data.
Collections

The sample_restaurants database contains the following collections:

Collection Name	Description
restaurants	Contains details on restaurants.

Collection Name	Description
neighborhoods	Contains details on neighborhoods.

sample_restaurants.restaurants

This collection contains account details for restaurants. Each document contains details on the restaurant such as its address, borough, review scores, its name, and the type of food it serves.
Indexes

The **sample_restaurants.restaurants** collection contains the following indexes:

Name	Index	Description
_id_	{ "_id": 1 }	Primary key index on the _id field.

Sample Document


```
{
"address": {
"building": "8825",
"coord": [-73.8803827, 40.7643124],
"street": "Astoria Boulevard",
"zipcode": "11369"
},
"borough": "Queens",
"cuisine": "American",
"grades": [ {
"date": {"$date": "2014-11-15T00:00:00.000Z"},
"grade": "Z",
"score": 38
},
{
"date": {"$date": "2014-05-02T00:00:00.000Z"},
"grade": "A",
"score": 10
},
{
"date": {"$date": "2013-03-02T00:00:00.000Z"},
"grade": "A",
```

"score": 7
},
{
"date": {"$date": "2012-02-10T00:00:00.000Z"},
"grade": "A",
"score": 13
}],
"name": "Brunos On The Boulevard",
"restaurant_id": "40356151"
}

sample_restaurants.neighborhoods

This collection contains details on the various neighborhoods of New York City neighborhoods. Each document contains the name of the neighborhood, and a geometry sub-document which contains the shape of the neighborhood.
These arrays of coordinates are typically used with the $geoWithin operator to query data that exists within a specified boundary.
Indexes

The sample_restaurants.neighborhoods collection contains the following indexes:

Name	Index	Description
_id_	{ "_id": 1 }	Primary key index on the _id field.

Sample Document

{
"geometry": {
"coordinates": [[
[-73.94193078816193, 40.70072523469547],
[-73.9443878859649, 40.70042452378256],
[-73.94424286147482, 40.69969927964773],
[-73.94409591260093, 40.69897295461309],
[-73.94394947271304, 40.69822127983908],

...
[-73.94193078816193, 40.70072523469547]
]]},
"name":"Bedford"
}

Source and solution can be found here:

http://www.w3resource.com/mongodb-exercises/

HINT

1. Create a Database and name it as exercises in MongoDB and create a Collection and name it as restaurants

```
use exercises

db.createCollection("restaurants")
```

2. Insert all the data given in the JSON file into this collection.

```
mongoimport --db exercises --collection restaurants –file d:\data\ restaurants.json
```

/* Linux:

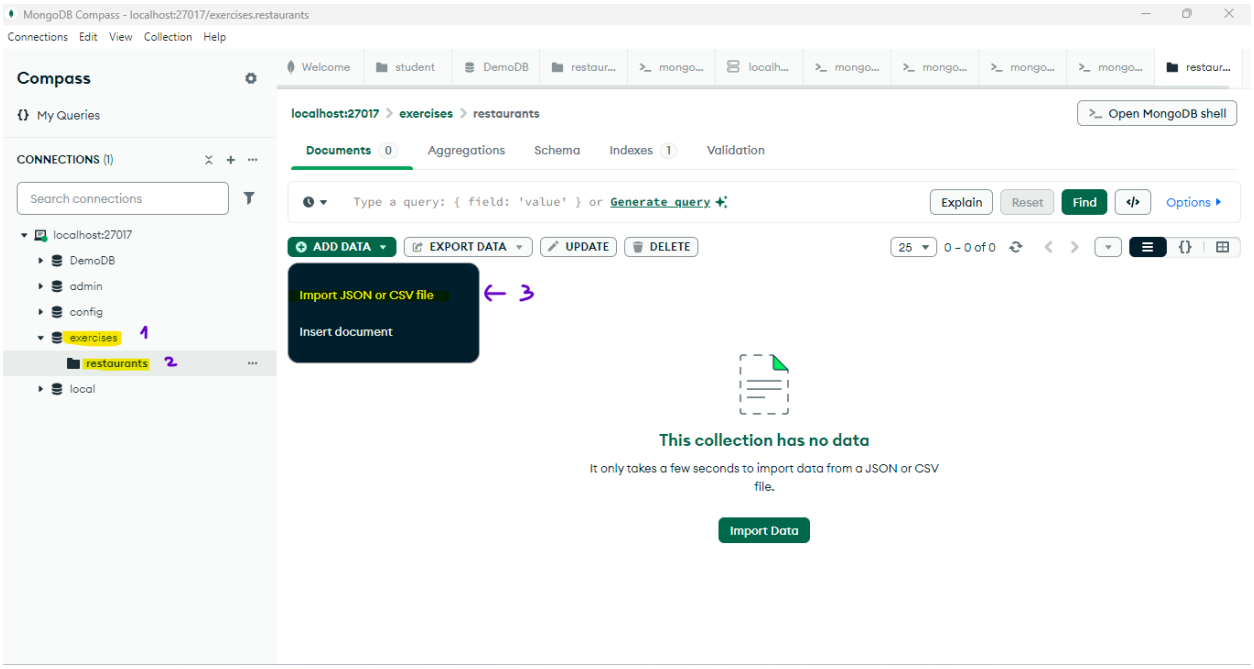
```
mongoimport --db exercises --collection restaurants --file /home/nilaxan/Downloads/UkiCohort8/restaurants.json */
```

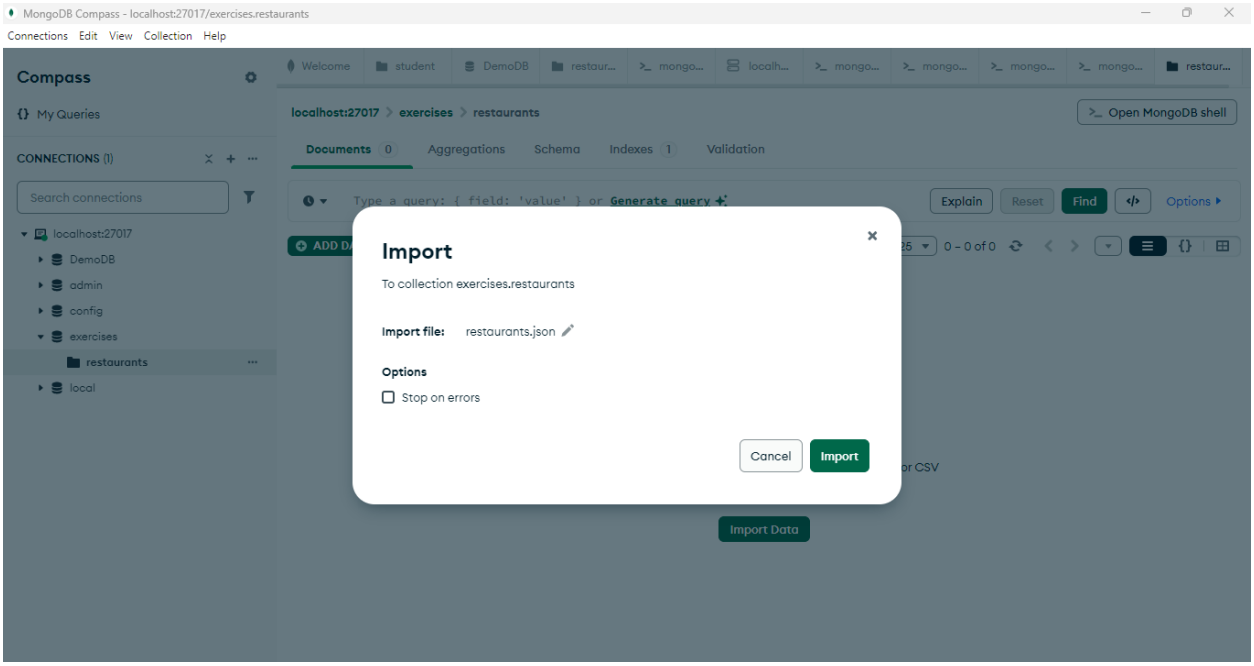
3. Write a MongoDB query to display all the documents in the collection restaurants.

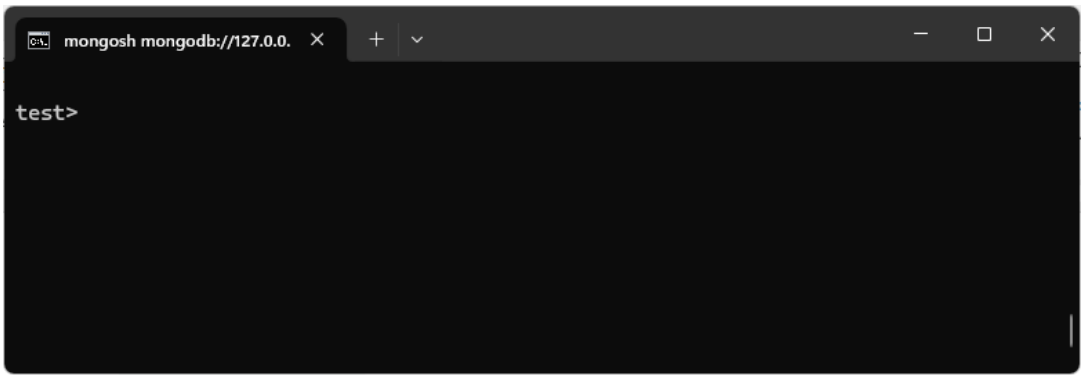
```
db.restaurants.find()
```

USING MONGODB COMPASS AND MONGODB SHELL

- Create a new database: `excercises`
- Create a new collection: `restaurants`
- Import data from `restaurants.json` into the `restaurants` collection
- Use Mongodb Shell to run queries


MongoDB Compass - localhost:27017/exercises.restaurants
Connections Edit View Collection Help
Compass
My Queries
CONNECTIONS (1)
Search connections
localhost:27017
DemoDB
admin
config
exercises
1
restaurants
2
local
Welcome
student
DemoDB
localhost:27017 > exercises > restaurants
Open MongoDB shell
Documents 0
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
EXPORT DATA
UPDATE
DELETE
25
0 - 0 of 0
Import JSON or CSV file
3
Insert document
This collection has no data
It only takes a few seconds to import data from a JSON or CSV file.
Import Data


MongoDB Compass - localhost:27017/exercises.restaurants
Connections Edit View Collection Help
Compass
My Queries
CONNECTIONS (1)
Search connections
localhost:27017
DemoDB
admin
config
exercises
restaurants
local
localhost:27017 > exercises > restaurants
Open MongoDB shell
Documents 0
Aggregations
Schema
Indexes 1
Validation
Explain
Reset
Find
Options
0 - 0 of 0
Import
To collection exercises.restaurants
Import file: restaurants.json
Options
Stop on errors
Cancel
Import
Import Data


mongosh mongodb://127.0.0.
test>

VI.2. MongoDB Exercises: Movies collection

Last update on December 01 2023 05:35:11 (UTC/GMT +8 hours)

MongoDB Query Exercises and Solution : Movie collection - 28 Exercises

Sample document of 'movies' collection:

```
{
    _id: ObjectId("573a1390f29313caabcd42e8"),
plot: 'A group of bandits stage a brazen train hold-up, only to find a
determined posse hot on their heels.',
genres: [ 'Short', 'Western' ],
runtime: 11,
cast: [
      'A.C. Abadie',
      "Gilbert M. 'Broncho Billy' Anderson",
      'George Barnes',
      'Justus D. Barnes'
    ],
poster: 'https://m.media-
amazon.com/images/M/MV5BMTU3NjE5NzYtYTYyNS00MDVmLWIwYjgtMmYwYWIxZDYyNzU2XkEy
XkFqcGdeQXVyNzQzNzQxNzI@._V1_SY1000_SX677_AL_.jpg',
title: 'The Great Train Robbery',
fullplot: "Among the earliest existing films in American cinema - notable as
the first film that presented a narrative story to tell - it depicts a group
of cowboy outlaws who hold up a train and rob the passengers. They are then
pursued by a Sheriff's posse. Several scenes have color included - all hand
tinted.",
languages: [ 'English' ],
released: ISODate("1903-12-01T00:00:00.000Z"),
directors: [ 'Edwin S. Porter' ],
rated: 'TV-G',
awards: { wins: 1, nominations: 0, text: '1 win.' },
lastupdated: '2015-08-13 00:27:59.177000000',
year: 1903,
imdb: { rating: 7.4, votes: 9847, id: 439 },
countries: [ 'USA' ],
type: 'movie',
tomatoes: {
viewer: { rating: 3.7, numReviews: 2559, meter: 75 },
fresh: 6,
critic: { rating: 7.6, numReviews: 6, meter: 100 },
rotten: 0,
lastUpdated: ISODate("2015-08-08T19:16:10.000Z")
    }
.....
```

(https://www.w3resource.com/mongodb-exercises/)

You may download the compressed file and uncompress it to find the collection used in our exercises. The collection comprises of 2000 documents.

1. Find all movies with full information from the 'movies' collection that released in the year 1893.

2. Find all movies with full information from the 'movies' collection that have a runtime greater than 120 minutes.

3. Find all movies with full information from the 'movies' collection that have "Short" genre.

4. Retrieve all movies from the 'movies' collection that were directed by "William K.L. Dickson" and include complete information for each movie.

5. Retrieve all movies from the 'movies' collection that were released in the USA and include complete information for each movie.

6. Retrieve all movies from the 'movies' collection that have complete information and are rated as "UNRATED".

7. Retrieve all movies from the 'movies' collection that have complete information and have received more than 1000 votes on IMDb.

8. Retrieve all movies from the 'movies' collection that have complete information and have an IMDb rating higher than 7.

9. Retrieve all movies from the 'movies' collection that have complete information and have a viewer rating higher than 4 on Tomatoes.

10. Retrieve all movies from the 'movies' collection that have received an award.

11. Find all movies with title, languages, released, directors, writers, awards, year, genres, runtime, cast, countries from the 'movies' collection in MongoDB that have at least one nomination.

12. Find all movies with title, languages, released, directors, writers, awards, year, genres, runtime, cast, countries from the 'movies' collection in MongoDB with cast including "Charles Kayser".

13. Retrieve all movies with title, languages, released, directors, writers, countries from the 'movies' collection in MongoDB that released on May 9, 1893.

14. Retrieve all movies with title, languages, released, directors, writers, countries from the 'movies' collection in MongoDB that have a word "scene" in the title.

15. Find all movies with title, languages, released, directors, viewer, writers, countries from the 'movies' collection in MongoDB that have a viewer rating of at least 3 and less than 4 on Tomatoes.

16. Retrieve all movies with title, languages, released, year, directors, writers, countries from the 'movies' collection in MongoDB that released before the year 1900.

17. Find all movies with title, languages, fullplot, released, directors, writers, countries from the 'movies' collection in MongoDB that have a fullplot containing the word "fire".

18. Return all movies with title, languages, plot, released, directors, writers, and countries from the 'movies' collection in MongoDB where the word "beer" mentioned in the plot.

19. Return all movies with title, languages, fullplot, released, directors, writers, and countries from the 'movies' collection in MongoDB where the word "metal" mentioned in the fullplot.

20. Find all movies with title, languages, released, runtime, directors, writers, countries from the 'movies' collection in MongoDB that have a runtime between 60 and 90 minutes.

21. Find all movies with title, languages, released, runtime, directors, writers, countries, imdb from the 'movies' collection in MongoDB for the top 5 movies with the highest IMDb ratings.

22. Find all movies from the 'movies' collection in MongoDB with the average runtime of movies released in each country.

23. Find from the 'movies' collection in MongoDB with the most common genre among the movies.

24. Find the movies released in the year with the highest average IMDb rating from the 'movies' collection in MongoDB.

25. Find the top 10 directors with the most movies from the 'movies' collection in MongoDB.

26. Write a query in MongoDB to find the average IMDb rating for movies with different ratings (e.g., 'PG', 'R', 'G') from the 'movies' collection.

27. Write a query in MongoDB to find the oldest movie with an award win from the 'movies' collection.

28. Write a query in MongoDB to find the movie with the highest IMDb rating and viewer rating on Tomatoes from the 'movies' collection.

Source and solution can be found here:

https://www.w3resource.com/mongodb-exercises/mongodb-movies-collection-index.php